ПР 25.09 - Логика службы каршеринга электрокаров

- **Предметная область:** платформа каршеринга электромобилей
- **СУБД:** Microsoft SQL Server 2025 (SSMS 22)
- **База данных:** CarSharingDB
- **Срок:** 25.09

Схема и тестовые данные

Схема и данные из задания. Даты со временем записаны в формате ISO (2025-09-17T09:00:00): при русском языке сервера строка «2025-09-17 09:00» для DATETIME читается как год-день-месяц и даёт ошибку 242.

Код

```
USE master;
GO
IF DB_ID(N'CarSharingDB') IS NOT NULL
BEGIN
    ALTER DATABASE CarSharingDB SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE CarSharingDB;
END
GO
CREATE DATABASE CarSharingDB;
GO
USE CarSharingDB;
GO
CREATE TABLE dbo.Cars (
    CarId           INT IDENTITY(1,1) PRIMARY KEY,
    VinNumber       NVARCHAR(50) NOT NULL UNIQUE,
    Model           NVARCHAR(50) NOT NULL,
    Year            INT NOT NULL,
    MaxRangeKm      INT NOT NULL,
    BatteryLevel    INT NOT NULL DEFAULT 100,
    CurrentMileage  DECIMAL(10,2) NOT NULL,
    Status          NVARCHAR(20) NOT NULL DEFAULT 'Available'
        CHECK (Status IN
('Available','Rented','Maintenance','Charging','Accident')),
    LastServiceDate DATE NULL,
    NextServiceKm   DECIMAL(8,2) NOT NULL
);
```

```sql
CREATE TABLE dbo.Drivers (
    DriverId      INT IDENTITY(1,1) PRIMARY KEY,
    FullName      NVARCHAR(100) NOT NULL,
    LicenseNumber NVARCHAR(20) NOT NULL UNIQUE,
    LicenseExpiry DATE NOT NULL,
    TotalRents    INT NOT NULL DEFAULT 0,
    TotalDistance DECIMAL(10,2) NOT NULL DEFAULT 0,
    AccidentCount INT NOT NULL DEFAULT 0,
    IsActive      BIT NOT NULL DEFAULT 1
);

CREATE TABLE dbo.Customers (
    CustomerId INT IDENTITY(1,1) PRIMARY KEY,
    FullName   NVARCHAR(100) NOT NULL,
    Phone      NVARCHAR(20) NOT NULL,
    Email      NVARCHAR(100) NULL,
    City       NVARCHAR(50) NOT NULL
);

CREATE TABLE dbo.Rentals (
    RentalId        INT IDENTITY(1,1) PRIMARY KEY,
    CustomerId      INT NOT NULL,
    CarId           INT NOT NULL,
    DriverId        INT NOT NULL,
    StartAt         DATETIME NOT NULL,
    EndAt           DATETIME NULL,
    StartOdometer   DECIMAL(10,2) NOT NULL,
    EndOdometer     DECIMAL(10,2) NULL,
    DistanceCovered DECIMAL(10,2) NULL,
    BaseCost        DECIMAL(10,2) NOT NULL,
    ExtraCost       DECIMAL(10,2) NOT NULL DEFAULT 0,
    TotalCost       DECIMAL(10,2) NULL,
    Status          NVARCHAR(20) NOT NULL DEFAULT 'Active'
        CHECK (Status IN ('Active','Completed','Cancelled','Disputed')),
    CreatedAt       DATETIME NOT NULL DEFAULT GETDATE(),
    CONSTRAINT FK_Rentals_Customer FOREIGN KEY (CustomerId) REFERENCES
dbo.Customers(CustomerId),
    CONSTRAINT FK_Rentals_Car FOREIGN KEY (CarId) REFERENCES dbo.Cars(CarId),
    CONSTRAINT FK_Rentals_Driver FOREIGN KEY (DriverId) REFERENCES
dbo.Drivers(DriverId)
);

CREATE TABLE dbo.ChargingLogs (
    LogId             INT IDENTITY(1,1) PRIMARY KEY,
    CarId             INT NOT NULL,
    ChargingStart     DATETIME NOT NULL,
```

```
    ChargingEnd       DATETIME NULL,
    EnergyConsumedKwh DECIMAL(6,2) NULL,
    ChargeFrom        INT NULL,
    ChargeTo          INT NULL,
    Location          NVARCHAR(100) NULL,
    CONSTRAINT FK_CL_Car FOREIGN KEY (CarId) REFERENCES dbo.Cars(CarId)
);

CREATE TABLE dbo.RentalStatusLog (
    LogId     INT IDENTITY(1,1) PRIMARY KEY,
    RentalId  INT NOT NULL,
    OldStatus NVARCHAR(20) NULL,
    NewStatus NVARCHAR(20) NOT NULL,
    ChangedAt DATETIME NOT NULL DEFAULT GETDATE(),
    ChangedBy NVARCHAR(100) NOT NULL DEFAULT SYSTEM_USER,
    CONSTRAINT FK_RSL_Rental FOREIGN KEY (RentalId) REFERENCES
dbo.Rentals(RentalId)
);
GO
INSERT dbo.Cars (VinNumber, Model, Year, MaxRangeKm, BatteryLevel,
CurrentMileage, Status, LastServiceDate, NextServiceKm) VALUES
(N'WBA12345678901234', N'BMW i3', 2021, 260, 82, 15200.5, N'Available', '2025-
08-01', 16000),
(N'WBA54321987654321', N'BMW i3', 2022, 260, 45, 9800.2, N'Available', '2025-
07-10', 10500),
(N'VIN-TESLA-001', N'Tesla Model 3', 2023, 528, 95, 3200.0, N'Available',
'2025-09-01', 5000),
(N'VIN-TESLA-002', N'Tesla Model 3', 2020, 425, 12, 41200.7, N'Charging',
'2025-08-15', 42000),
(N'VIN-NISSAN-LEAF', N'Nissan Leaf', 2019, 240, 70, 58000.3, N'Maintenance',
'2025-06-01', 60000);

INSERT dbo.Drivers (FullName, LicenseNumber, LicenseExpiry, TotalRents,
TotalDistance, AccidentCount, IsActive) VALUES
(N'Игорь Петров', N'DL-2024-101', '2027-12-31', 35, 1200.5, 0, 1),
(N'Ольга Смирнова', N'DL-2024-102', '2026-11-20', 52, 2100.0, 1, 1),
(N'Дмитрий Волков', N'DL-2023-103', '2025-10-10', 8, 320.2, 2, 1),
(N'Анна Кузнецова', N'DL-2025-104', '2028-03-05', 12, 450.0, 0, 1);

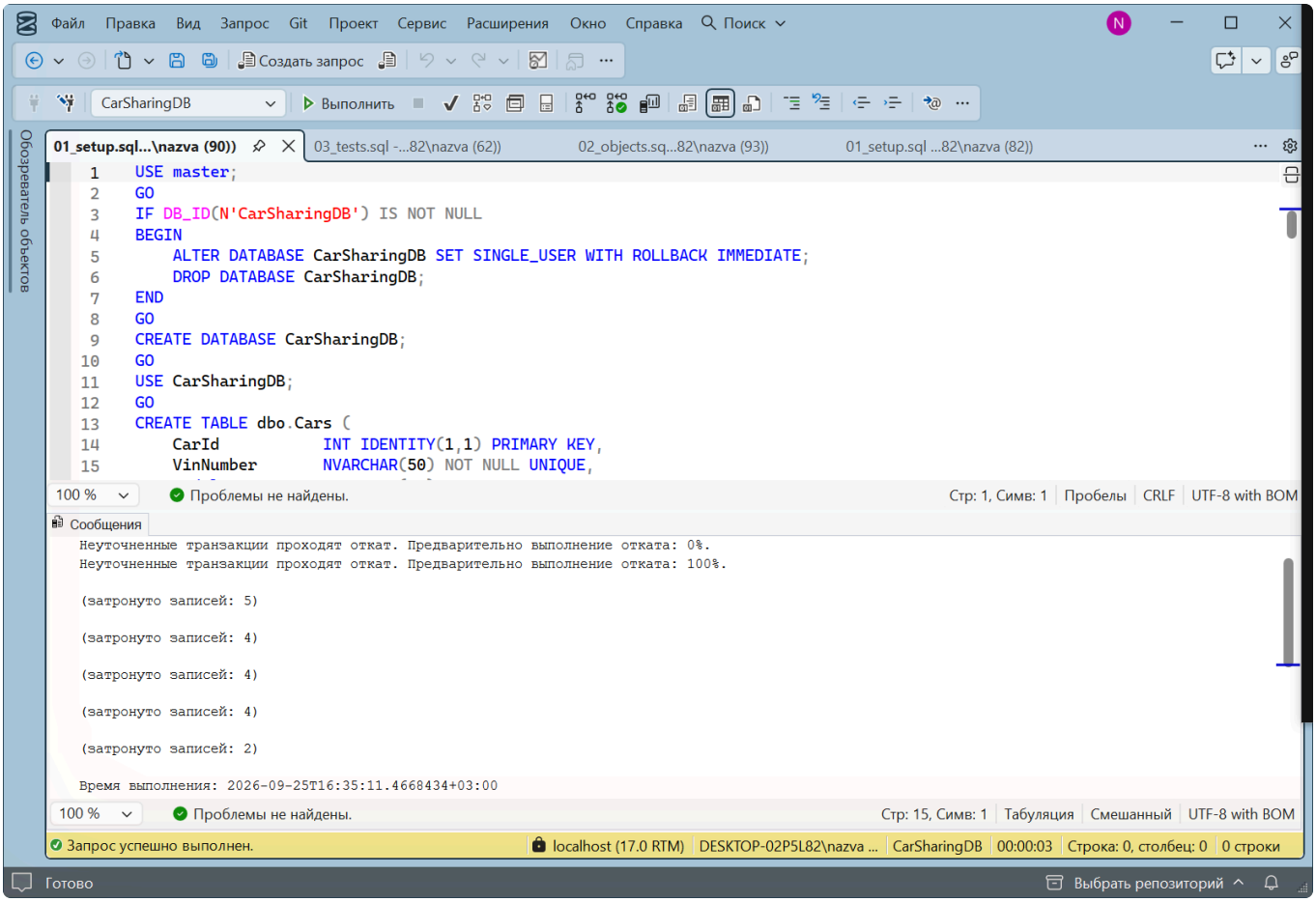
INSERT dbo.Customers (FullName, Phone, Email, City) VALUES
(N'Сергей Иванов', N'+7-900-111-22-33', N'sergey@example.com', N'Москва'),
(N'Елена Морозова', N'+7-900-222-33-44', N'elena@example.com', N'Москва'),
(N'Максим Соколов', N'+7-900-333-44-55', NULL, N'Санкт-Петербург'),
(N'Ирина Лебедева', N'+7-900-444-55-66', N'irina@example.com', N'Казань');

INSERT dbo.Rentals (CustomerId, CarId, DriverId, StartAt, EndAt,
```

```
StartOdometer, EndOdometer, DistanceCovered, BaseCost, ExtraCost, TotalCost,
Status, CreatedAt) VALUES
(1, 1, 1, '2025-09-17T09:00:00', '2025-09-17T14:30:00', 15200.5, 15245.2,
44.7, 2800.00, 0, NULL, 'Completed', '2025-09-17T08:55:00'),
(2, 3, 2, '2025-09-17T11:00:00', '2025-09-17T13:15:00', 3200.0, 3285.5, 85.5,
4200.00, 0, NULL, 'Completed', '2025-09-17T10:55:00'),
(3, 2, 3, '2025-09-18T08:00:00', NULL, 9800.2, NULL, NULL, 1800.00, 0, NULL,
'Active', '2025-09-18T07:55:00'),
(4, 1, 4, '2025-09-18T10:30:00', NULL, 15245.2, NULL, NULL, 2500.00, 0, NULL,
'Active', '2025-09-18T10:25:00');

INSERT dbo.ChargingLogs (CarId, ChargingStart, ChargingEnd, EnergyConsumedKwh,
ChargeFrom, ChargeTo, Location) VALUES
(2, '2025-09-17T20:00:00', '2025-09-18T00:30:00', 32.5, 45, 98, N'Зарядная
станция ВДНХ'),
(4, '2025-09-18T09:00:00', NULL, NULL, 12, NULL, N'Подземная парковка ТЦ');
```

Скриншот 1. Создание и заполнение CarSharingDB



Скриншот 2. Создание всех объектов

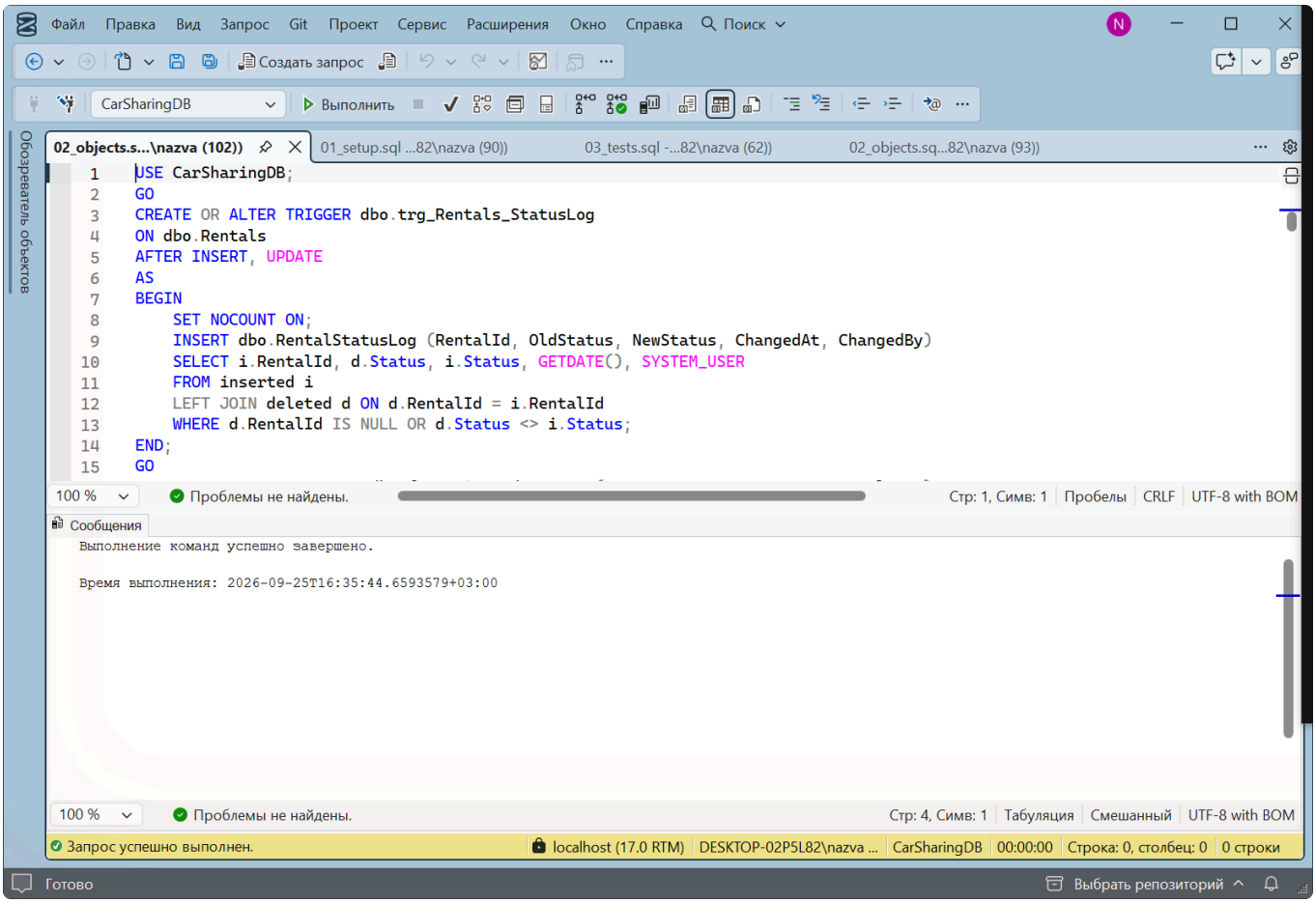


Таблица обоснований

№	Выбранный объект	Почему этот, а не другой?	Почему альтернативы не подходят?
1	Триггер AFTER INSERT, UPDATE на Rentals (trg_Rentals_StatusLog)	Журнал должен заполняться при любом изменении статуса: из процедуры, прямым UPDATE, из приложения. Триггер срабатывает всегда и видит старое (deleted) и новое (inserted) значение	Процедуру можно обойти прямым UPDATE, и оператор забудет запись. Функция не может делать INSERT
2	Скалярная функция fn_EstimatedRangeKm	Это расчёт, который нужен внутри SELECT, WHERE, отчётов и других объектов	Процедуру нельзя вызвать внутри SELECT. Триггер не возвращает значение, а хранить

№	Выбранный объект	Почему этот, а не другой?	Почему альтернативы не подходят?
		(используется в StartRental)	вычисляемое значение в таблице - дублирование
3	Хранимая процедура StartRental	Действие с параметрами, несколькими проверками, транзакцией и понятными ошибками, меняет две таблицы (Rentals и Cars)	Функция не может менять данные. Триггер срабатывает на уже сделанное изменение и не принимает параметры операции
4	Хранимая процедура CompleteRental	Многошаговая атомарная операция: Rentals, Cars, Drivers в одной транзакции, с проверками и THROW	Функция не может делать UPDATE. В триггере сложно передать параметры (пробег, заряд, авария) и управлять ошибками
5	Триггер AFTER UPDATE на Cars (trg_Cars_LowBatteryToCharging)	Перевод в Charging должен происходить автоматически при любом снижении заряда, не только из CompleteRental	Процедура сработает только если её вызвать. Функция не меняет данные
6	Скалярная функция fn_DriverReliability	Расчёт показателя для SELECT и отчётов по водителям	Процедура не встраивается в SELECT. Триггер и хранимый столбец дублируют данные и требуют синхронизации
7	Хранимая процедура FinishCharging	Оператор фиксирует окончание зарядки с параметрами, нужны проверки (ChargeTo >= ChargeFrom), атомарное	Функция не может менять данные. Триггер на ChargingLogs не проверит входные параметры до изменения и хуже сообщает об ошибке

№	Выбранный объект	Почему этот, а не другой?	Почему альтернативы не подходят?
		обновление ChargingLogs и Cars	
8	Триггер AFTER INSERT, UPDATE на Rentals (trg_Rentals_BlockUnavailableCar)	Гарантия на уровне БД: проверка срабатывает при любом способе назначить авто, в том числе прямом UPDATE	Проверку в процедуре обходит прямой UPDATE или приложение. CHECK-ограничение не может обращаться к другой таблице без функции, а функция в CHECK не даёт понятного текста ошибки

Требование 1. Автоматический журнал статусов аренды

Код

```
CREATE OR ALTER TRIGGER dbo.trg_Rentals_StatusLog
ON dbo.Rentals
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    INSERT dbo.RentalStatusLog (RentalId, OldStatus, NewStatus, ChangedAt,
ChangedBy)
    SELECT i.RentalId, d.Status, i.Status, GETDATE(), SYSTEM_USER
    FROM inserted i
    LEFT JOIN deleted d ON d.RentalId = i.RentalId
    WHERE d.RentalId IS NULL OR d.Status <> i.Status;
END;
```

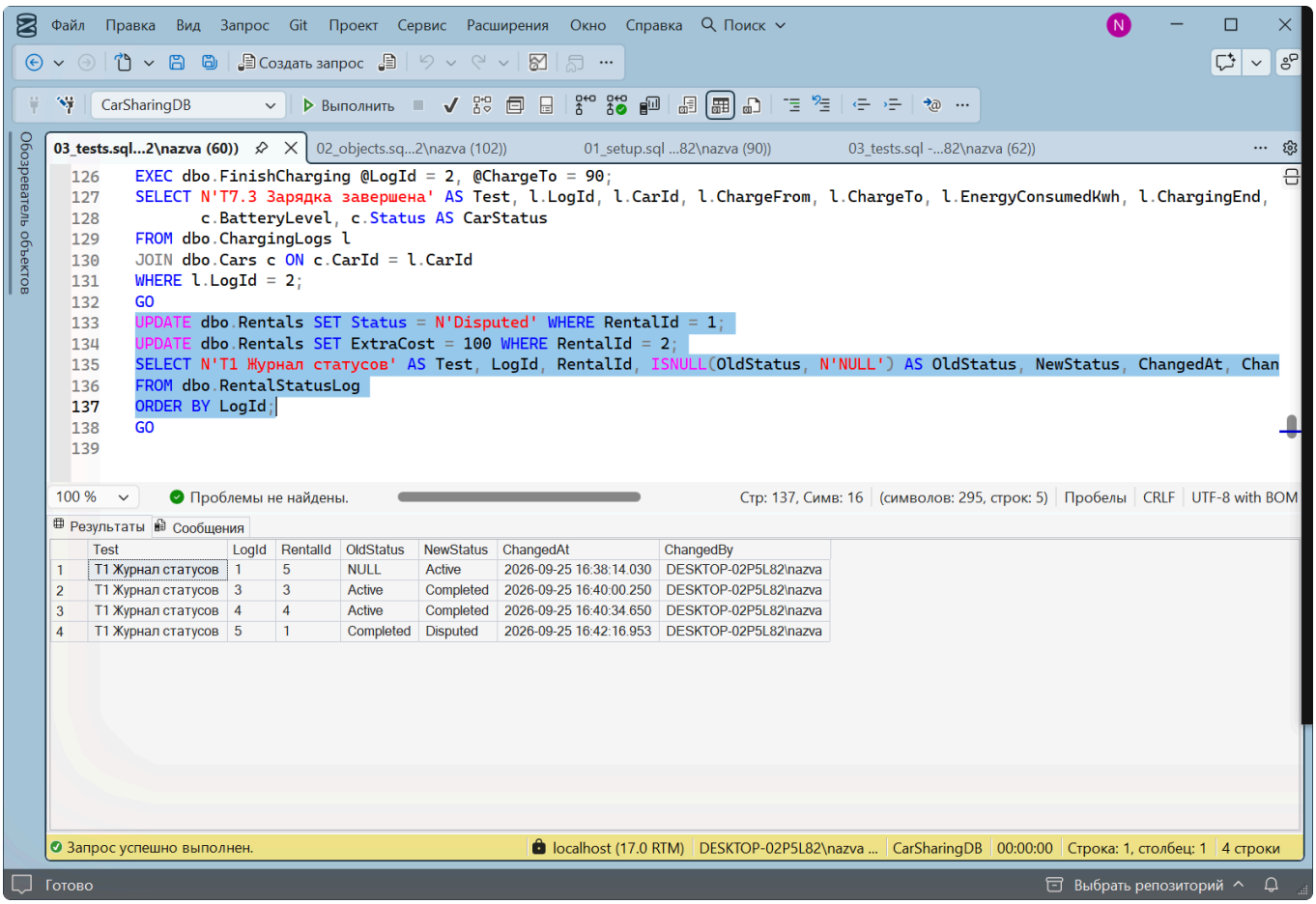
Тест

```
UPDATE dbo.Rentals SET Status = N'Disputed' WHERE RentalId = 1;
UPDATE dbo.Rentals SET ExtraCost = 100 WHERE RentalId = 2;
SELECT N'T1 Журнал статусов' AS Test, LogId, RentalId, ISNULL(OldStatus,
N'NULL') AS OldStatus, NewStatus, ChangedAt, ChangedBy
```

```
FROM dbo.RentalStatusLog
ORDER BY LogId;
```

Результат: Смена статуса аренды 1 на Disputed и изменение ExtraCost у аренды 2 без смены статуса. В журнале 4 записи: NULL -> Active (новая аренда 5), Active -> Completed (аренды 3 и 4), Completed -> Disputed (аренда 1), ChangedBy = DESKTOP-02P5L82\nazva. Изменение ExtraCost в журнал не попало.

Скриншот 23. Журнал статусов RentalStatusLog



Требование 2. Расчёт примерного запаса хода

Код

```
CREATE OR ALTER FUNCTION dbo.fn_EstimatedRangeKm (@MaxRangeKm INT,
@BatteryLevel INT)
RETURNS DECIMAL(8,2)
AS
BEGIN
    IF @MaxRangeKm IS NULL OR @BatteryLevel IS NULL OR @MaxRangeKm <= 0 OR
@BatteryLevel NOT BETWEEN 0 AND 100
        RETURN NULL;
```

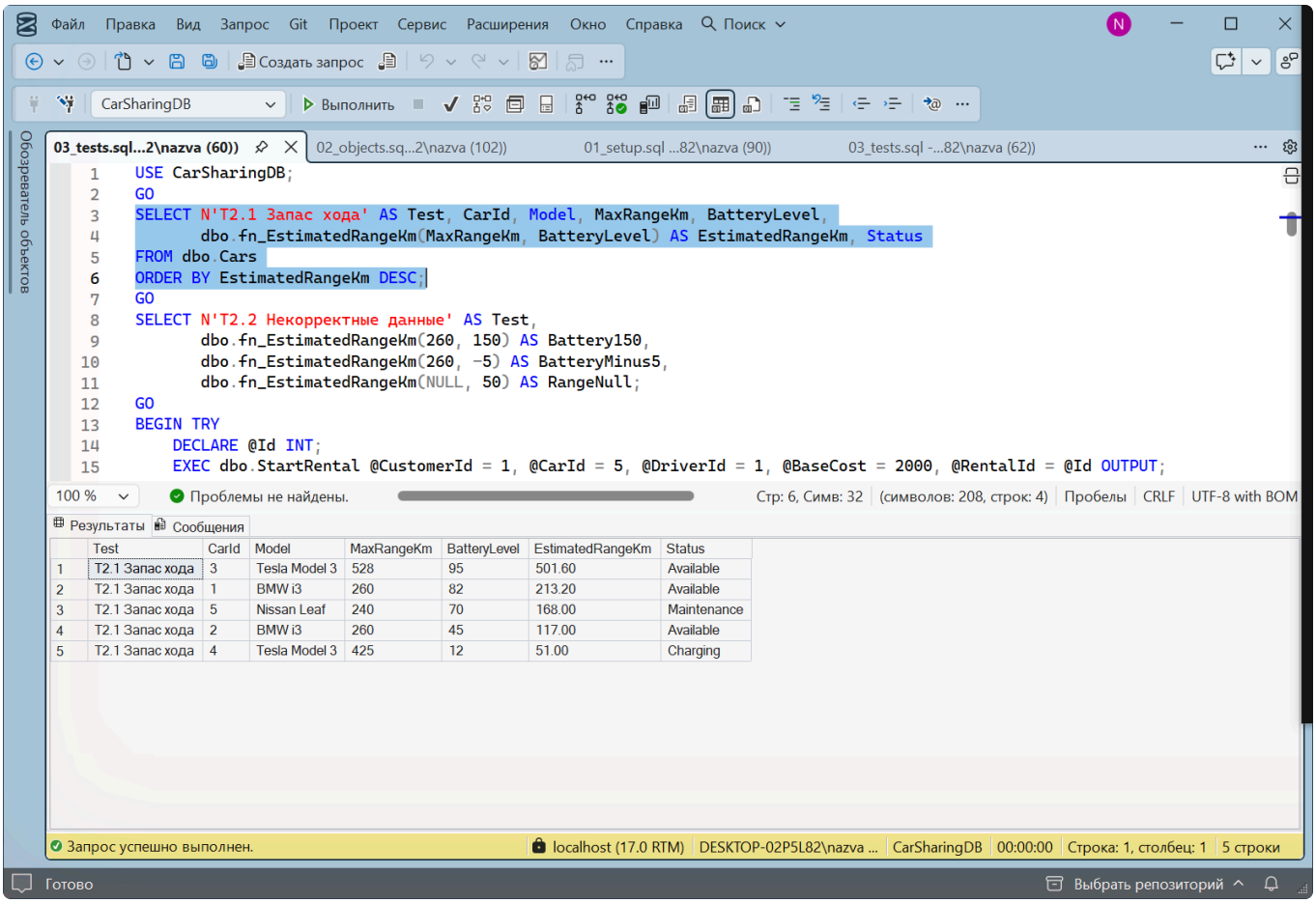
```
    RETURN CAST(@MaxRangeKm * (@BatteryLevel / 100.0) AS DECIMAL(8,2));
END;
```

Тест

```
SELECT N'T2.1 Запас хода' AS Test, CarId, Model, MaxRangeKm, BatteryLevel,
       dbo.fn_EstimatedRangeKm(MaxRangeKm, BatteryLevel) AS EstimatedRangeKm,
Status
FROM dbo.Cars
ORDER BY EstimatedRangeKm DESC;
```

Результат: Запас хода: Tesla Model 3 (CarId 3) 501.60 км, BMW i3 (CarId 1) 213.20, Nissan Leaf 168.00, BMW i3 (CarId 2) 117.00, Tesla Model 3 (CarId 4) 51.00.

Скриншот 3. Запас хода по всем авто



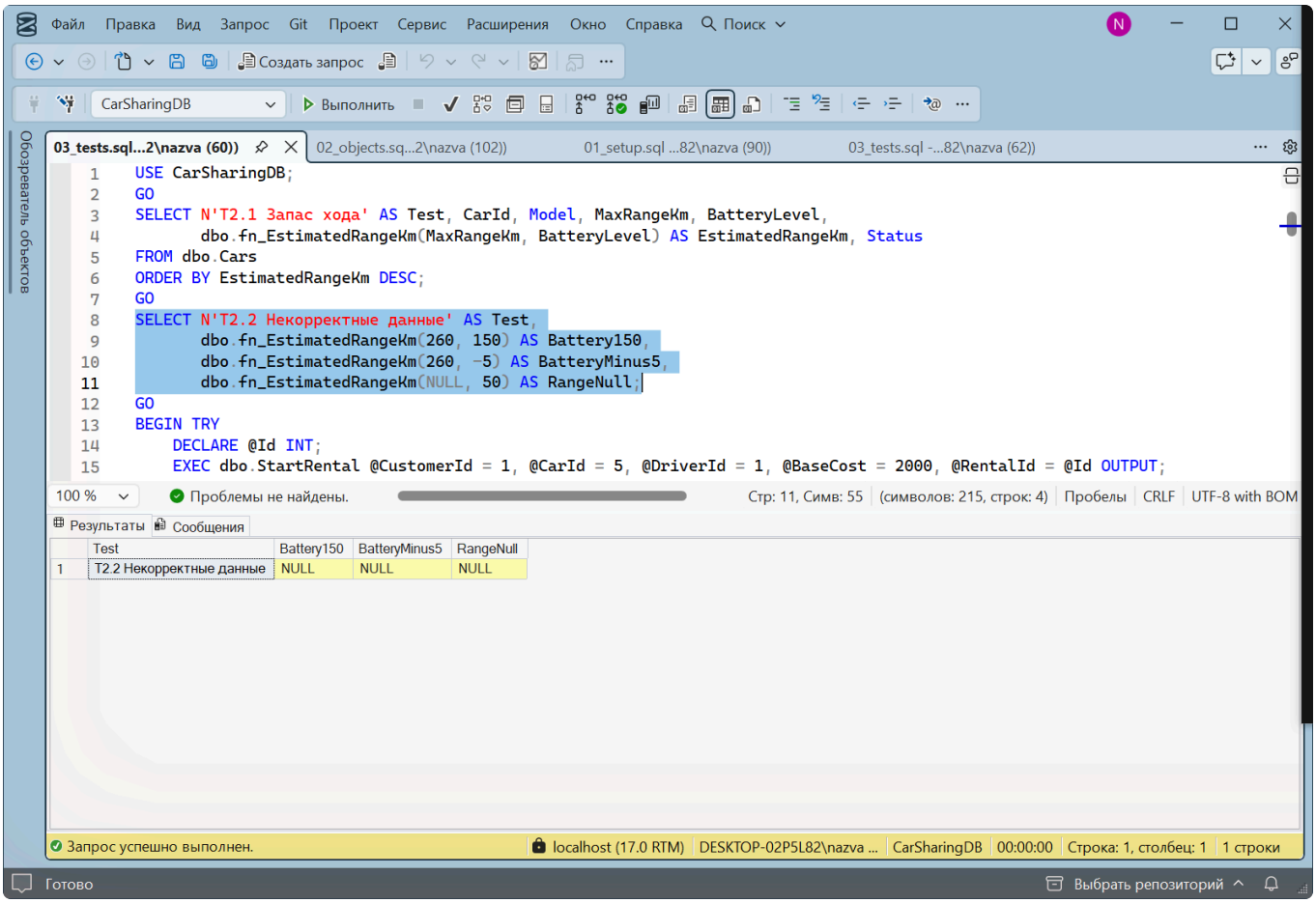
Тест

```
SELECT N'T2.2 Некорректные данные' AS Test,
       dbo.fn_EstimatedRangeKm(260, 150) AS Battery150,
```

```
        dbo.fn_EstimatedRangeKm(260, -5) AS BatteryMinus5,
        dbo.fn_EstimatedRangeKm(NULL, 50) AS RangeNull;
```

Результат: Некорректные входные данные (заряд 150, -5, пустой запас хода) - функция возвращает NULL.

Скриншот 4. Обработка некорректных значений



Требование 3. Оформление аренды авто

Код

```
CREATE OR ALTER PROCEDURE dbo.StartRental
    @CustomerId INT,
    @CarId      INT,
    @DriverId   INT,
    @BaseCost   DECIMAL(10,2),
    @RentalId   INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;
    DECLARE @Msg NVARCHAR(400), @Model NVARCHAR(50), @CarStatus NVARCHAR(20),
```

```
@MaxRange INT, @Battery INT,
            @Mileage DECIMAL(10,2), @DriverName NVARCHAR(100), @Expiry DATE,
@DriverActive BIT, @Range DECIMAL(8,2);
    BEGIN TRY
        BEGIN TRANSACTION;
            IF NOT EXISTS (SELECT 1 FROM dbo.Customers WHERE CustomerId =
@CustomerId)
            BEGIN
                SET @Msg = CONCAT(N'Клиент с CustomerId = ', @CustomerId, N'
не найден.');
                THROW 52301, @Msg, 1;
            END;

            SELECT @Model = Model, @CarStatus = Status, @MaxRange =
MaxRangeKm, @Battery = BatteryLevel, @Mileage = CurrentMileage
            FROM dbo.Cars WITH (UPDLOCK, HOLDLOCK)
            WHERE CarId = @CarId;

            IF @Model IS NULL
            BEGIN
                SET @Msg = CONCAT(N'Автомобиль с CarId = ', @CarId, N' не
найден.');
                THROW 52302, @Msg, 1;
            END;
            IF @CarStatus <> N'Available'
            BEGIN
                SET @Msg = CONCAT(N'Автомобиль ', @Model, N' (CarId = ',
@CarId, N') недоступен: статус ', @CarStatus, N'.');
                THROW 52303, @Msg, 1;
            END;
            IF EXISTS (SELECT 1 FROM dbo.Rentals WHERE CarId = @CarId AND
Status = N'Active')
            BEGIN
                SET @Msg = CONCAT(N'Автомобиль ', @Model, N' (CarId = ',
@CarId, N') уже находится в активной аренде.');
                THROW 52304, @Msg, 1;
            END;

            SELECT @DriverName = FullName, @Expiry = LicenseExpiry,
@DriverActive = IsActive
            FROM dbo.Drivers
            WHERE DriverId = @DriverId;

            IF @DriverName IS NULL OR @DriverActive = 0
            BEGIN
                SET @Msg = CONCAT(N'Водитель с DriverId = ', @DriverId, N' не
```

```
найден или неактивен.');
                THROW 52305, @Msg, 1;
            END;
            IF @Expiry < CAST(GETDATE() AS DATE)
            BEGIN
                SET @Msg = CONCAT(N'У водителя ', @DriverName, N' истекла
лицензия (', CONVERT(NVARCHAR(10), @Expiry, 104), N').');
                THROW 52306, @Msg, 1;
            END;

            SET @Range = dbo.fn_EstimatedRangeKm(@MaxRange, @Battery);
            IF @Range < @MaxRange * 0.3
            BEGIN
                SET @Msg = CONCAT(N'Недостаточно заряда: запас хода ', @Range,
N' км при минимуме ', CAST(@MaxRange * 0.3 AS DECIMAL(8,2)), N' км (заряд ',
@Battery, N' проц.).');
                THROW 52307, @Msg, 1;
            END;
            IF @BaseCost IS NULL OR @BaseCost <= 0
            BEGIN
                SET @Msg = N'Базовая стоимость аренды должна быть больше
нуля.';
                THROW 52308, @Msg, 1;
            END;

            INSERT dbo.Rentals (CustomerId, CarId, DriverId, StartAt,
StartOdometer, BaseCost, Status)
            VALUES (@CustomerId, @CarId, @DriverId, GETDATE(), @Mileage,
@BaseCost, N'Active');
            SET @RentalId = SCOPE_IDENTITY();

            UPDATE dbo.Cars SET Status = N'Rented' WHERE CarId = @CarId;
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH;
END;
```

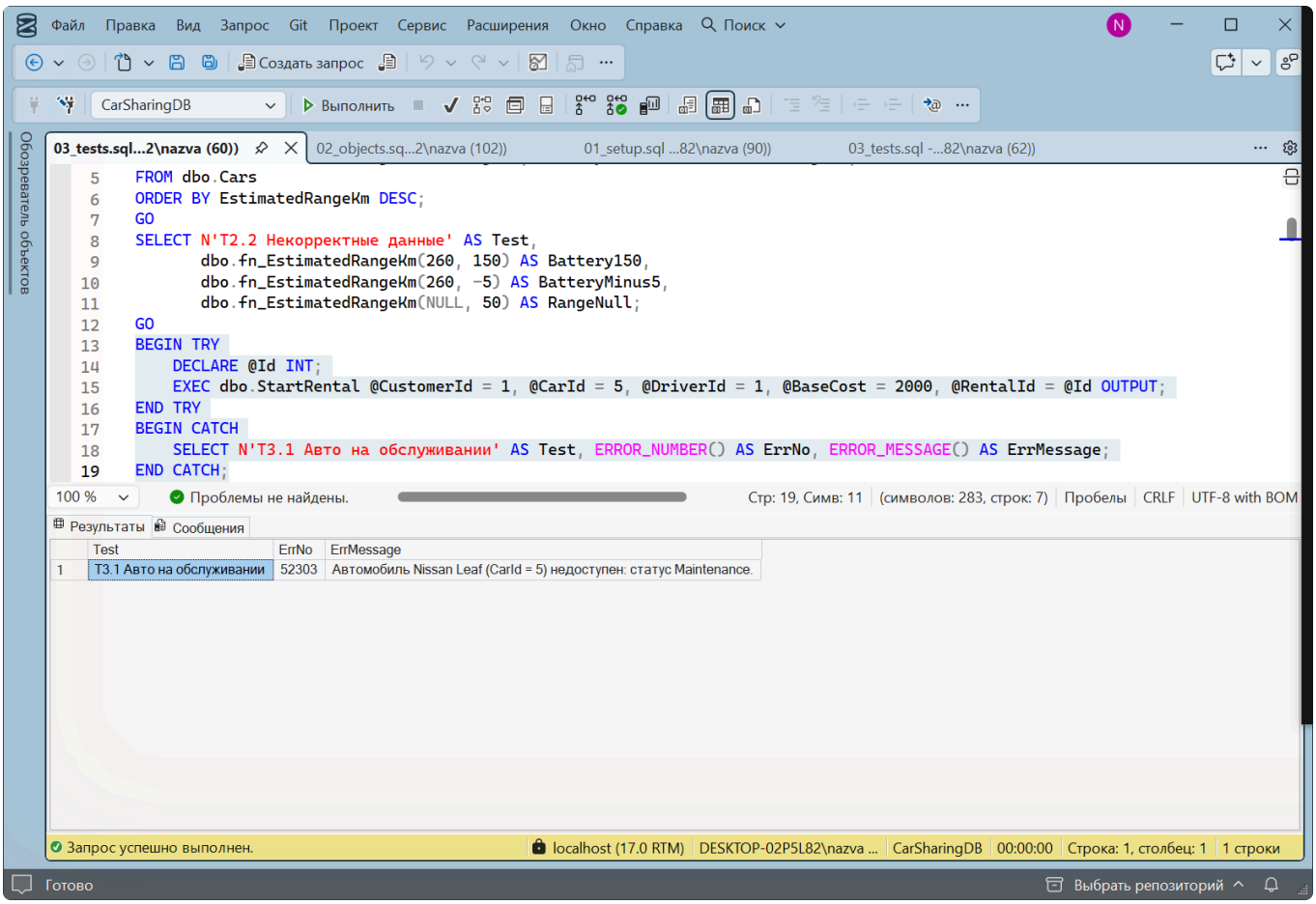
Тест

```
BEGIN TRY
    DECLARE @Id INT;
    EXEC dbo.StartRental @CustomerId = 1, @CarId = 5, @DriverId = 1, @BaseCost
```

```
= 2000, @RentalId = @Id OUTPUT;
END TRY
BEGIN CATCH
    SELECT N'T3.1 Авто на обслуживании' AS Test, ERROR_NUMBER() AS ErrNo,
ERROR_MESSAGE() AS ErrMessage;
END CATCH;
```

Результат: Ошибка 52303 «Автомобиль Nissan Leaf (CarId = 5) недоступен: статус Maintenance.».

Скриншот 5. Авто на обслуживании



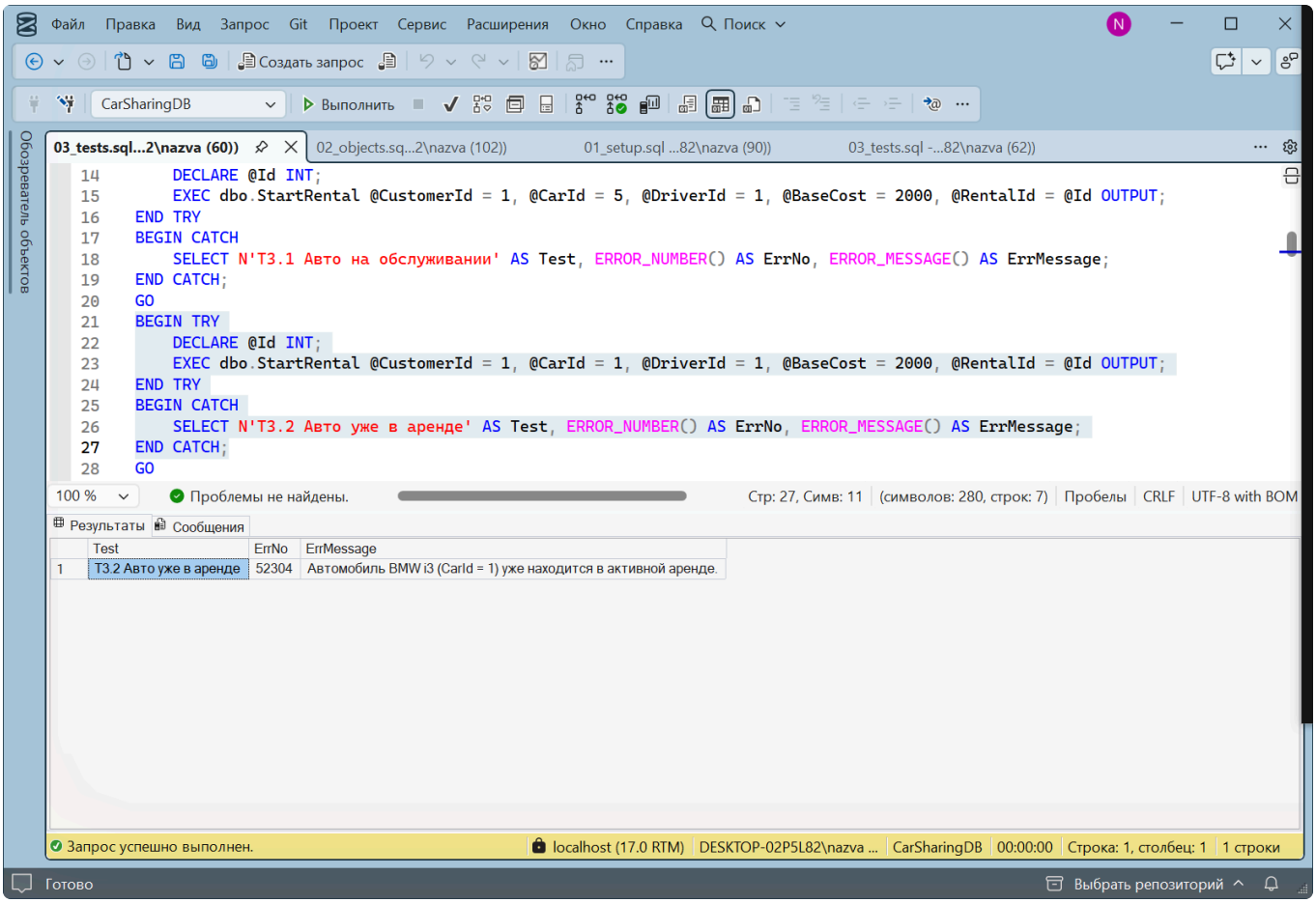
Тест

```
BEGIN TRY
    DECLARE @Id INT;
    EXEC dbo.StartRental @CustomerId = 1, @CarId = 1, @DriverId = 1, @BaseCost
= 2000, @RentalId = @Id OUTPUT;
END TRY
BEGIN CATCH
    SELECT N'T3.2 Авто уже в аренде' AS Test, ERROR_NUMBER() AS ErrNo,
```

```
ERROR_MESSAGE() AS ErrMessage;
END CATCH;
```

Результат: Ошибка 52304 «Автомобиль BMW i3 (CarId = 1) уже находится в активной аренде.» (в данных у авто 1 статус Available, но есть активная аренда 4).

Скриншот 6. Авто уже в аренде

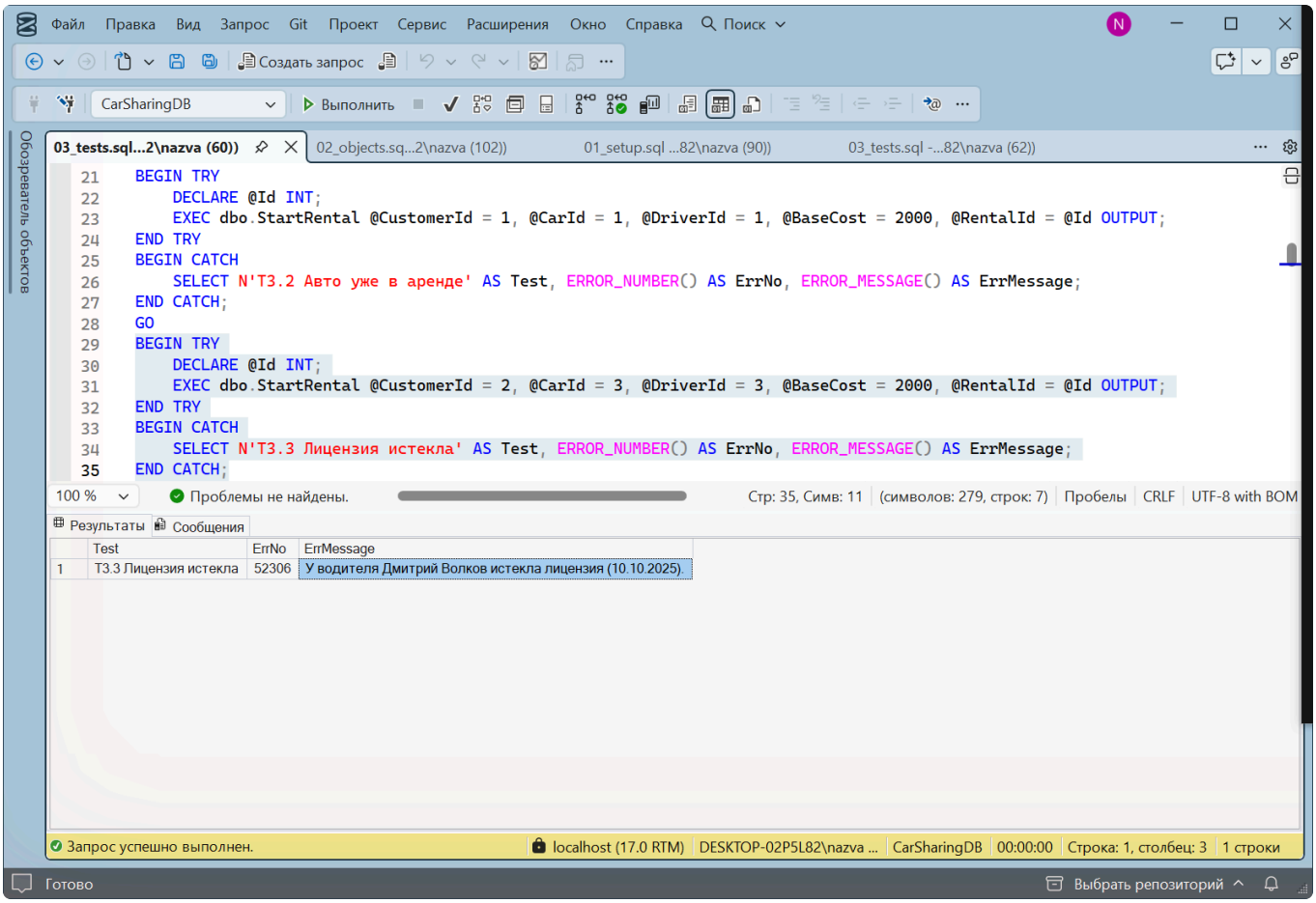


Тест

```
BEGIN TRY
    DECLARE @Id INT;
    EXEC dbo.StartRental @CustomerId = 2, @CarId = 3, @DriverId = 3, @BaseCost
= 2000, @RentalId = @Id OUTPUT;
END TRY
BEGIN CATCH
    SELECT N'T3.3 Лицензия истекла' AS Test, ERROR_NUMBER() AS ErrNo,
ERROR_MESSAGE() AS ErrMessage;
END CATCH;
```

Результат: Ошибка 52306 «У водителя Дмитрий Волков истекла лицензия (10.10.2025).».

Скриншот 7. Истекшая лицензия

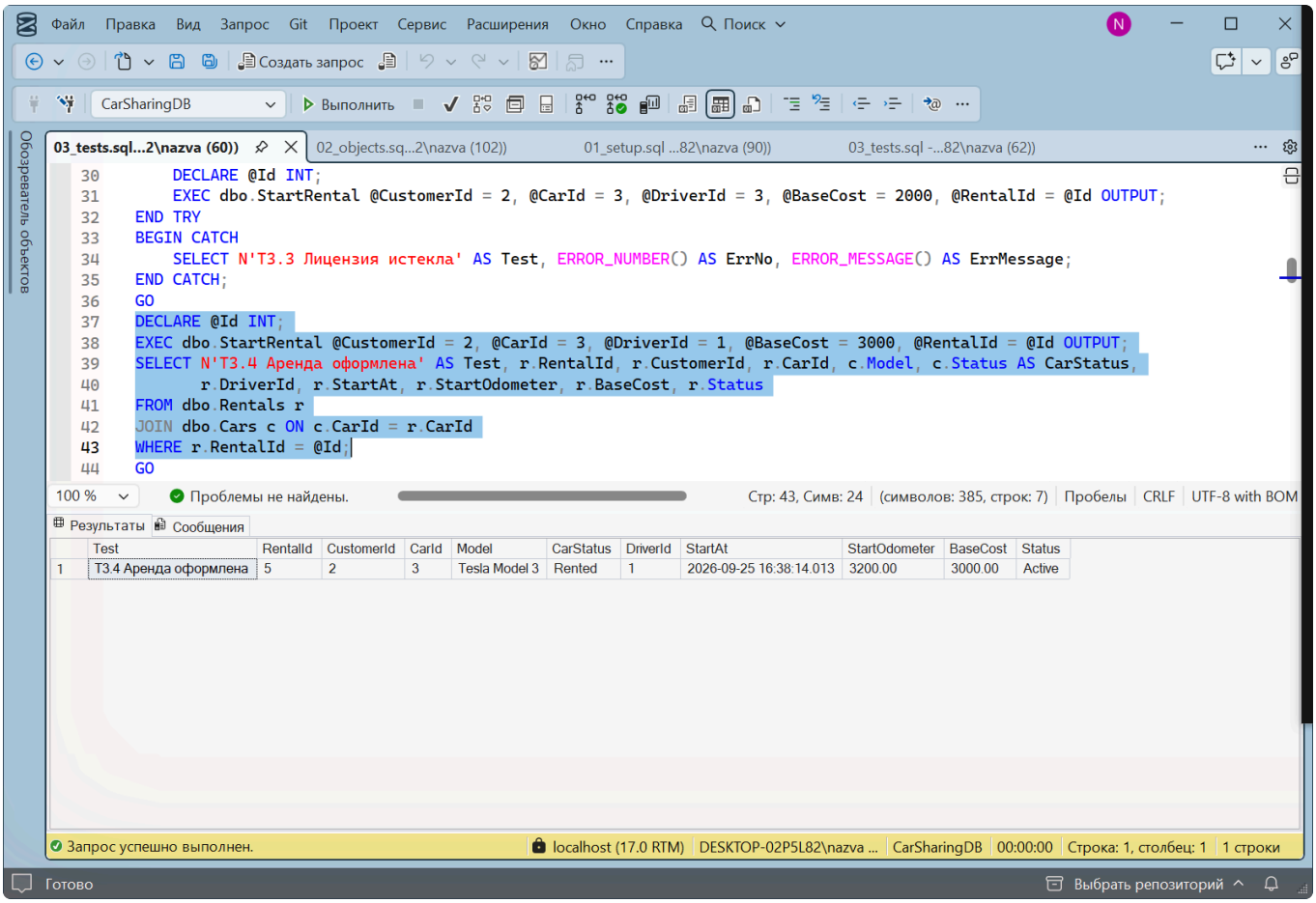


Тест

```
DECLARE @Id INT;
EXEC dbo.StartRental @CustomerId = 2, @CarId = 3, @DriverId = 1, @BaseCost =
3000, @RentalId = @Id OUTPUT;
SELECT N'T3.4 Аренда оформлена' AS Test, r.RentalId, r.CustomerId, r.CarId,
c.Model, c.Status AS CarStatus,
       r.DriverId, r.StartAt, r.StartOdometer, r.BaseCost, r.Status
FROM dbo.Rentals r
JOIN dbo.Cars c ON c.CarId = r.CarId
WHERE r.RentalId = @Id;
```

Результат: Аренда 5 оформлена: Tesla Model 3 (CarId 3), водитель 1, StartOdometer 3200.00, статус авто Rented.

Скриншот 8. Успешное оформление аренды

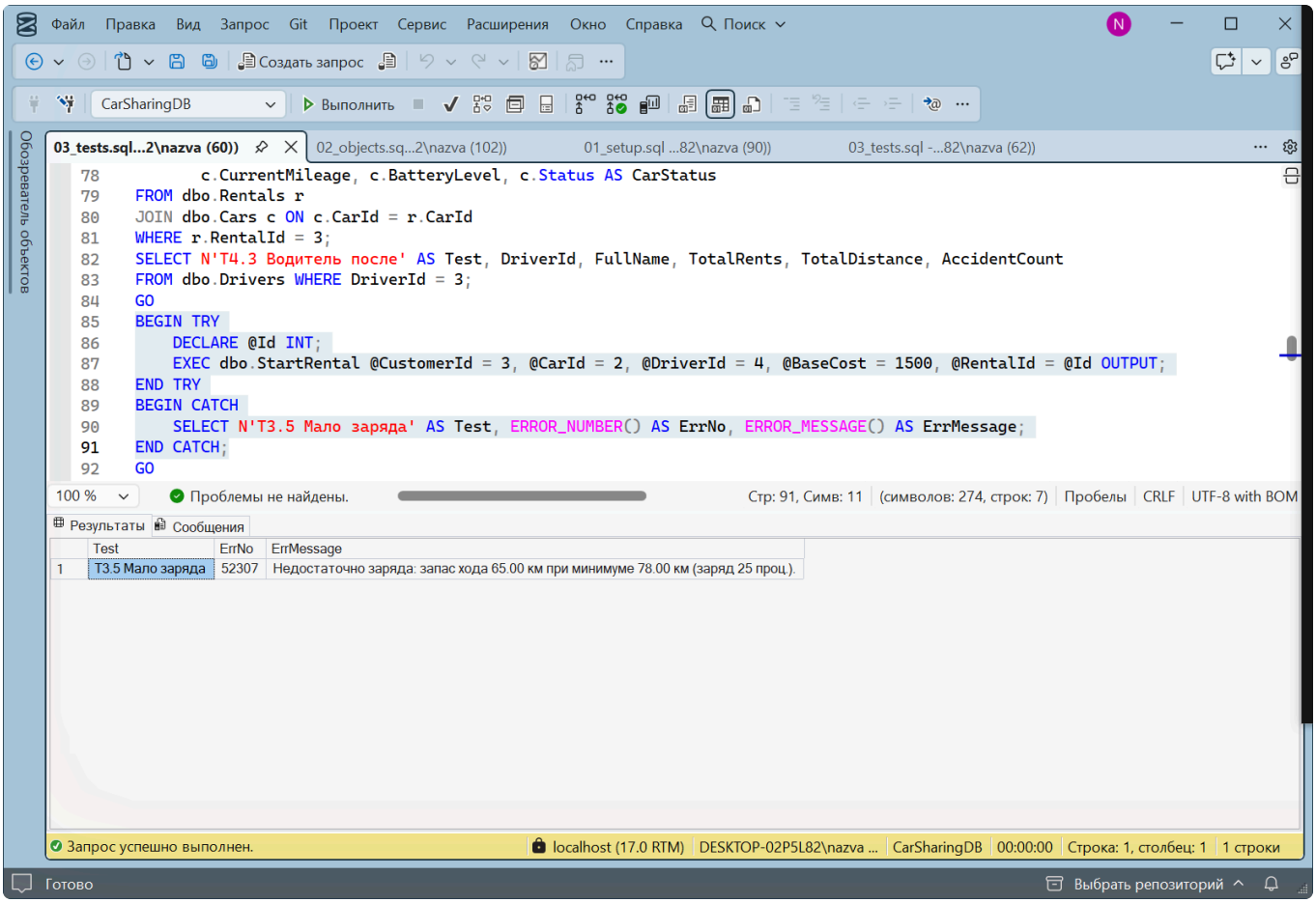


Тест

```
BEGIN TRY
    DECLARE @Id INT;
    EXEC dbo.StartRental @CustomerId = 3, @CarId = 2, @DriverId = 4, @BaseCost
= 1500, @RentalId = @Id OUTPUT;
END TRY
BEGIN CATCH
    SELECT N'T3.5 Мало заряда' AS Test, ERROR_NUMBER() AS ErrNo,
ERROR_MESSAGE() AS ErrMessage;
END CATCH;
```

Результат: После аренды 3 у BMW i3 (CarId 2) заряд 25. Ошибка 52307 «Недостаточно заряда: запас хода 65.00 км при минимуме 78.00 км (заряд 25 проц.).».

Скриншот 15. Недостаточно заряда



Требование 4. Фиксация завершения аренды и расчёт стоимости

Код

```
CREATE OR ALTER PROCEDURE dbo.CompleteRental
    @RentalId         INT,
    @EndOdometer      DECIMAL(10,2),
    @EndBatteryLevel  INT,
    @ExtraCost        DECIMAL(10,2) = 0,
    @HadAccident      BIT = 0
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;
    DECLARE @Msg NVARCHAR(400), @Status NVARCHAR(20), @CarId INT, @DriverId
INT,
            @StartOdo DECIMAL(10,2), @Distance DECIMAL(10,2);
    BEGIN TRY
        BEGIN TRANSACTION;
            SELECT @Status = Status, @CarId = CarId, @DriverId = DriverId,
@StartOdo = StartOdometer
            FROM dbo.Rentals WITH (UPDLOCK, HOLDLOCK)
```

```
            WHERE RentalId = @RentalId;

            IF @Status IS NULL
            BEGIN
                SET @Msg = CONCAT(N'Аренда с RentalId = ', @RentalId, N' не
найдена.');
                THROW 52401, @Msg, 1;
            END;
            IF @Status <> N'Active'
            BEGIN
                SET @Msg = CONCAT(N'Аренда ', @RentalId, N' имеет статус ',
@Status, N'. Закрыть можно только активную аренду.');
                THROW 52402, @Msg, 1;
            END;
            IF @EndOdometer IS NULL OR @EndOdometer < @StartOdo
            BEGIN
                SET @Msg = CONCAT(N'Конечный пробег ', @EndOdometer, N' меньше
начального ', @StartOdo, N'.');
                THROW 52403, @Msg, 1;
            END;
            IF @EndBatteryLevel IS NULL OR @EndBatteryLevel NOT BETWEEN 0 AND
100
            BEGIN
                SET @Msg = CONCAT(N'Некорректный уровень заряда: ',
@EndBatteryLevel, N'.');
                THROW 52404, @Msg, 1;
            END;
            IF @ExtraCost < 0
            BEGIN
                SET @Msg = N'Доплата не может быть отрицательной.';
                THROW 52405, @Msg, 1;
            END;

            SET @Distance = @EndOdometer - @StartOdo;

            UPDATE dbo.Rentals
            SET EndAt = GETDATE(),
                EndOdometer = @EndOdometer,
                DistanceCovered = @Distance,
                ExtraCost = @ExtraCost,
                TotalCost = BaseCost + @ExtraCost,
                Status = N'Completed'
            WHERE RentalId = @RentalId;

            UPDATE dbo.Cars
            SET CurrentMileage = @EndOdometer,
```

```
                BatteryLevel = @EndBatteryLevel,
                Status = CASE WHEN @HadAccident = 1 THEN N'Accident' ELSE
N'Available' END
            WHERE CarId = @CarId;

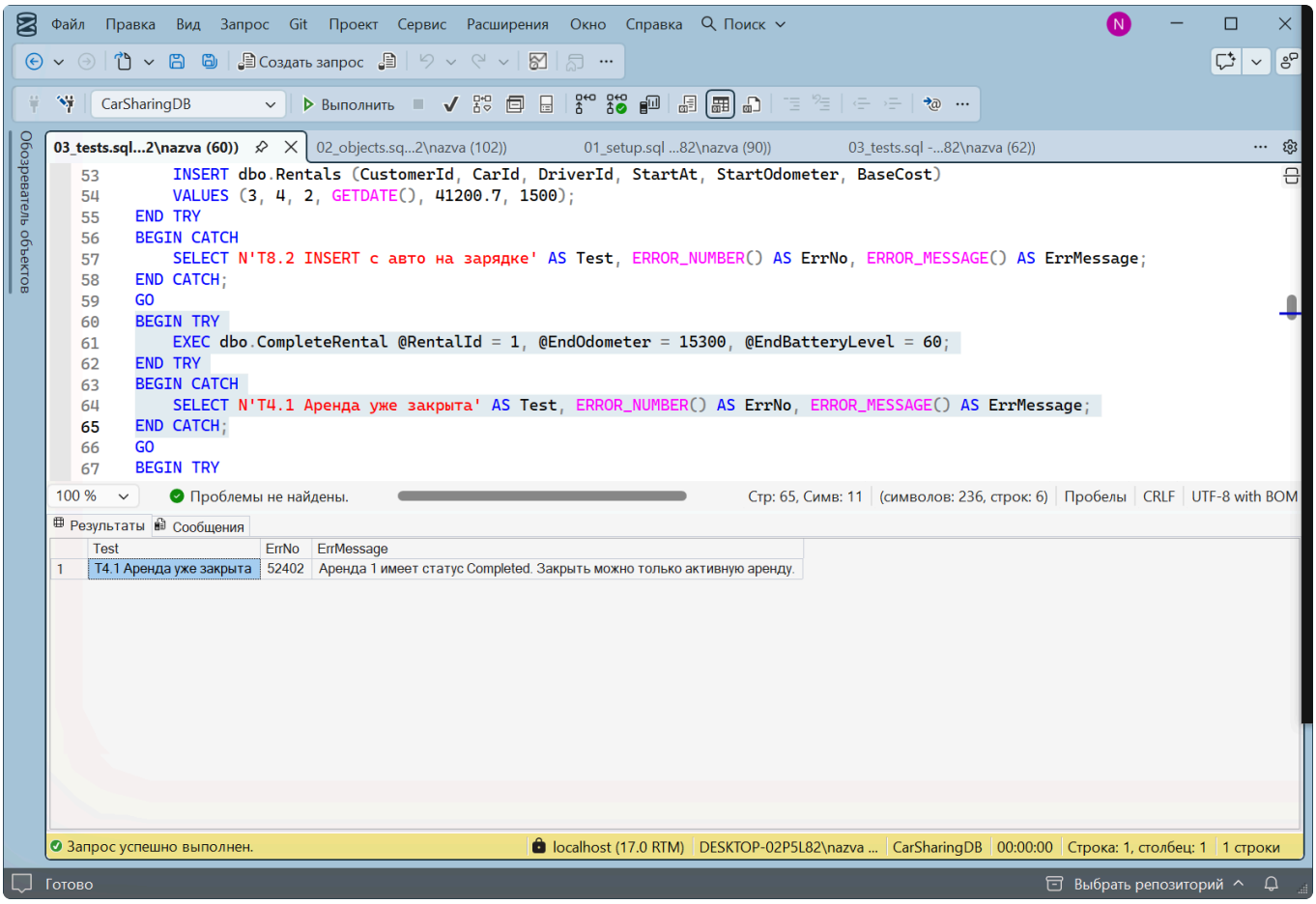
            UPDATE dbo.Drivers
            SET TotalRents = TotalRents + 1,
                TotalDistance = TotalDistance + @Distance,
                AccidentCount = AccidentCount + CASE WHEN @HadAccident = 1
THEN 1 ELSE 0 END
            WHERE DriverId = @DriverId;
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH;
END;
```

Тест

```
BEGIN TRY
    EXEC dbo.CompleteRental @RentalId = 1, @EndOdometer = 15300,
@EndBatteryLevel = 60;
END TRY
BEGIN CATCH
    SELECT N'T4.1 Аренда уже закрыта' AS Test, ERROR_NUMBER() AS ErrNo,
ERROR_MESSAGE() AS ErrMessage;
END CATCH;
```

Результат: Ошибка 52402 «Аренда 1 имеет статус Completed. Закрыть можно только активную аренду.».

Скриншот 12. Аренда уже закрыта

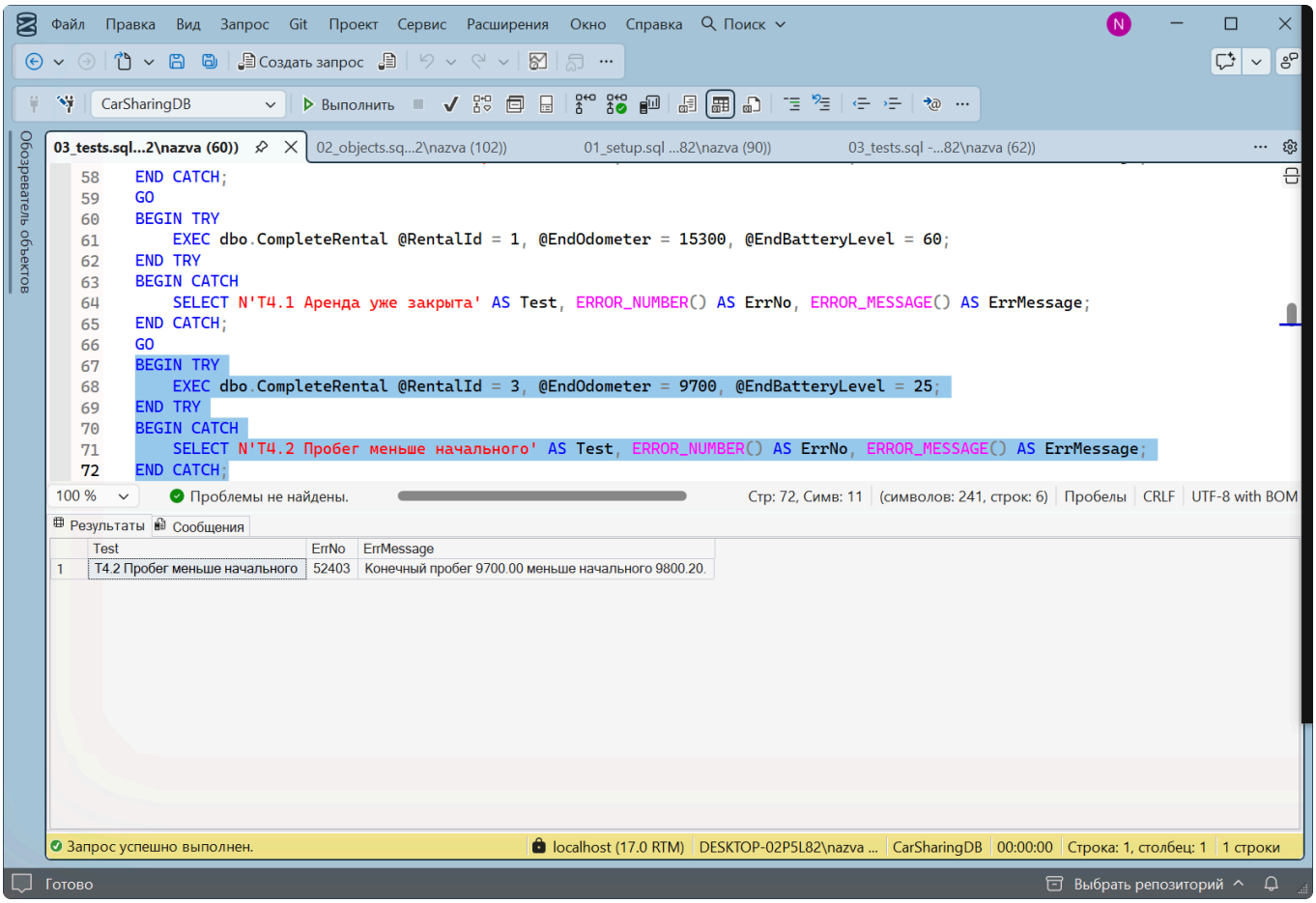


Тест

```
BEGIN TRY
    EXEC dbo.CompleteRental @RentalId = 3, @EndOdometer = 9700,
@EndBatteryLevel = 25;
END TRY
BEGIN CATCH
    SELECT N'T4.2 Пробег меньше начального' AS Test, ERROR_NUMBER() AS ErrNo,
ERROR_MESSAGE() AS ErrMessage;
END CATCH;
```

Результат: Ошибка 52403 «Конечный пробег 9700.00 меньше начального 9800.20.».

Скриншот 13. Пробег меньше начального

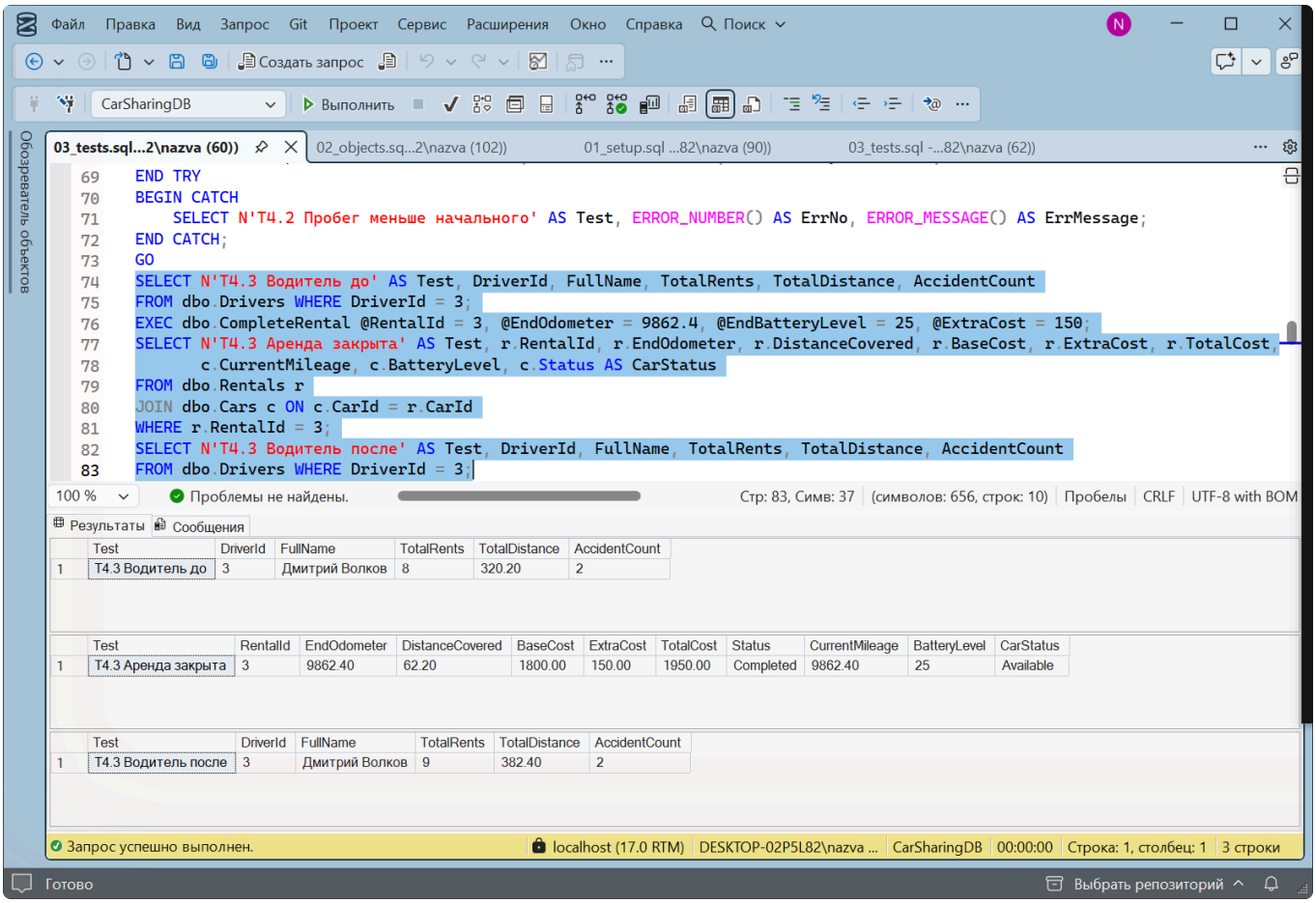


Тест

```
SELECT N'T4.3 Водитель до' AS Test, DriverId, FullName, TotalRents,
TotalDistance, AccidentCount
FROM dbo.Drivers WHERE DriverId = 3;
EXEC dbo.CompleteRental @RentalId = 3, @EndOdometer = 9862.4, @EndBatteryLevel
= 25, @ExtraCost = 150;
SELECT N'T4.3 Аренда закрыта' AS Test, r.RentalId, r.EndOdometer,
r.DistanceCovered, r.BaseCost, r.ExtraCost, r.TotalCost, r.Status,
       c.CurrentMileage, c.BatteryLevel, c.Status AS CarStatus
FROM dbo.Rentals r
JOIN dbo.Cars c ON c.CarId = r.CarId
WHERE r.RentalId = 3;
SELECT N'T4.3 Водитель после' AS Test, DriverId, FullName, TotalRents,
TotalDistance, AccidentCount
FROM dbo.Drivers WHERE DriverId = 3;
```

Результат: Аренда 3 закрыта: DistanceCovered 62.20, TotalCost 1950.00 (1800 + 150), пробег авто 9862.40, заряд 25, статус Available. У водителя Дмитрий Волков TotalRents 8 -> 9, TotalDistance 320.20 -> 382.40.

Скриншот 14. Закрытие аренды и обновление водителя



Требование 5. Авто-перевод в зарядку при низком заряде

Код

```
CREATE OR ALTER TRIGGER dbo.trg_Cars_LowBatteryToCharging
ON dbo.Cars
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF NOT UPDATE(BatteryLevel) RETURN;

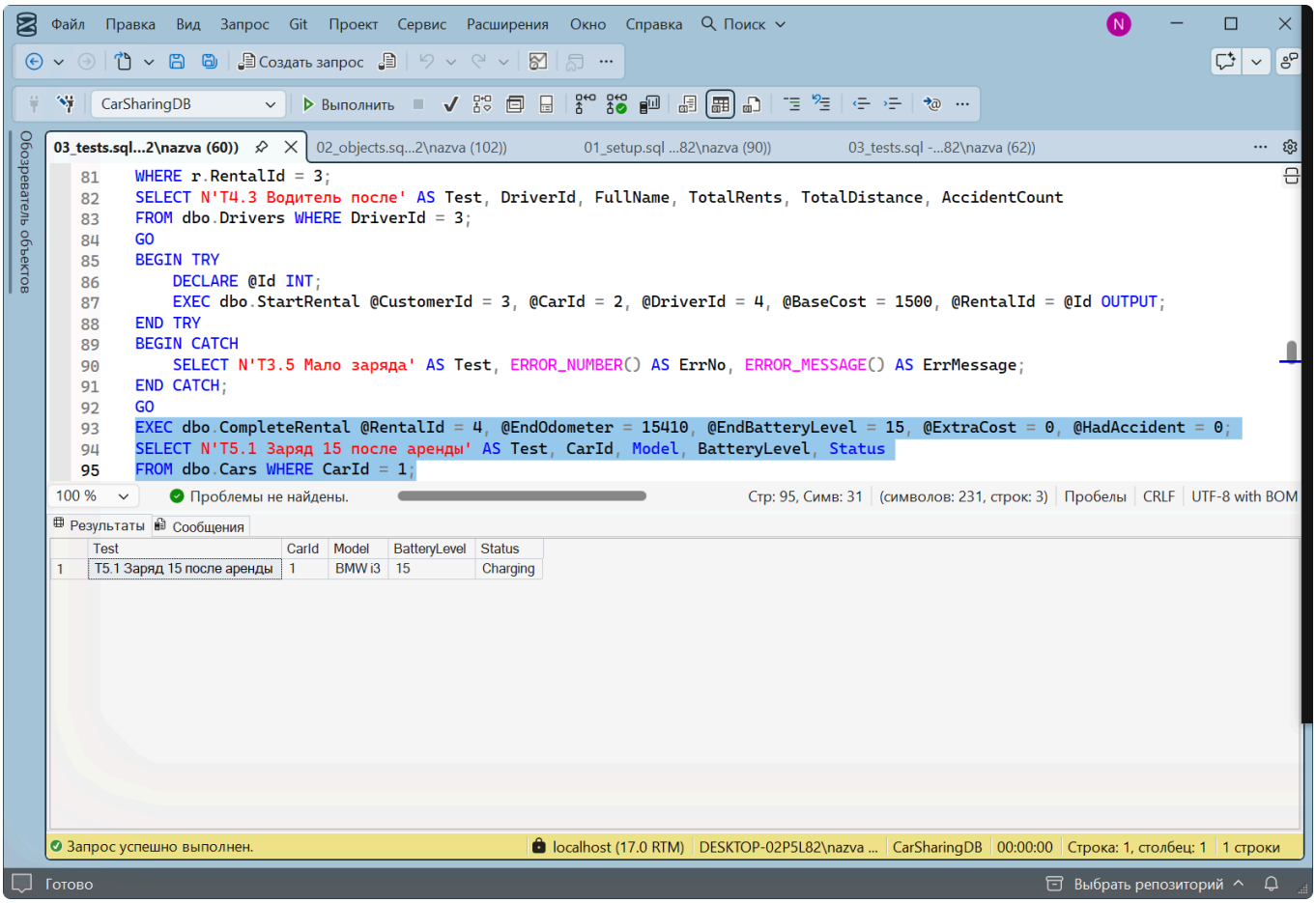
    UPDATE c
    SET c.Status = N'Charging'
    FROM dbo.Cars c
    JOIN inserted i ON i.CarId = c.CarId
    WHERE i.BatteryLevel < 20
      AND i.Status = N'Available';
END;
```

Тест

```
EXEC dbo.CompleteRental @RentalId = 4, @EndOdometer = 15410, @EndBatteryLevel
= 15, @ExtraCost = 0, @HadAccident = 0;
SELECT N'T5.1 Заряд 15 после аренды' AS Test, CarId, Model, BatteryLevel,
Status
FROM dbo.Cars WHERE CarId = 1;
```

Результат: Аренда 4 закрыта с зарядом 15 - BMW i3 (CarId 1) автоматически переведён в Charging.

Скриншот 16. Автоперевод в Charging

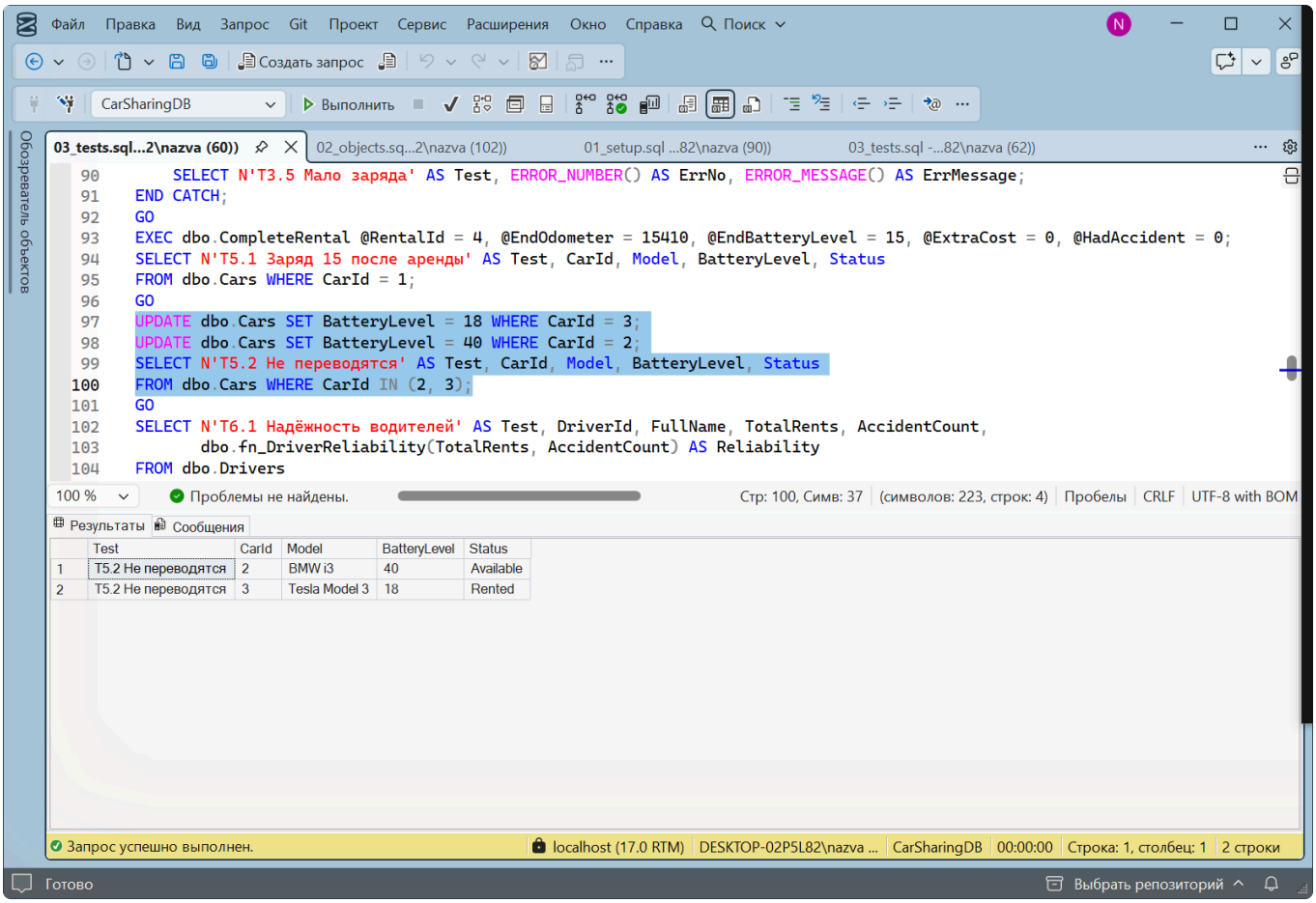


Тест

```
UPDATE dbo.Cars SET BatteryLevel = 18 WHERE CarId = 3;
UPDATE dbo.Cars SET BatteryLevel = 40 WHERE CarId = 2;
SELECT N'T5.2 Не переводятся' AS Test, CarId, Model, BatteryLevel, Status
FROM dbo.Cars WHERE CarId IN (2, 3);
```

Результат: Заряд 40 у свободного авто - статус Available не меняется. Заряд 18 у авто в аренде (Rented) - статус не меняется, перевод только для свободных авто.

Скриншот 17. Случаи без перевода



Требование 6. Рейтинг надёжности водителя

Код

```
CREATE OR ALTER FUNCTION dbo.fn_DriverReliability (@TotalRents INT,
@AccidentCount INT)
RETURNS DECIMAL(5,2)
AS
BEGIN
    DECLARE @Result DECIMAL(5,2);
    IF @TotalRents IS NULL OR @TotalRents <= 0
        RETURN 0;
    SET @Result = (@TotalRents - ISNULL(@AccidentCount, 0)) * 100.0 /
@TotalRents;
    RETURN CASE WHEN @Result < 0 THEN 0 WHEN @Result > 100 THEN 100 ELSE
@Result END;
END;
```

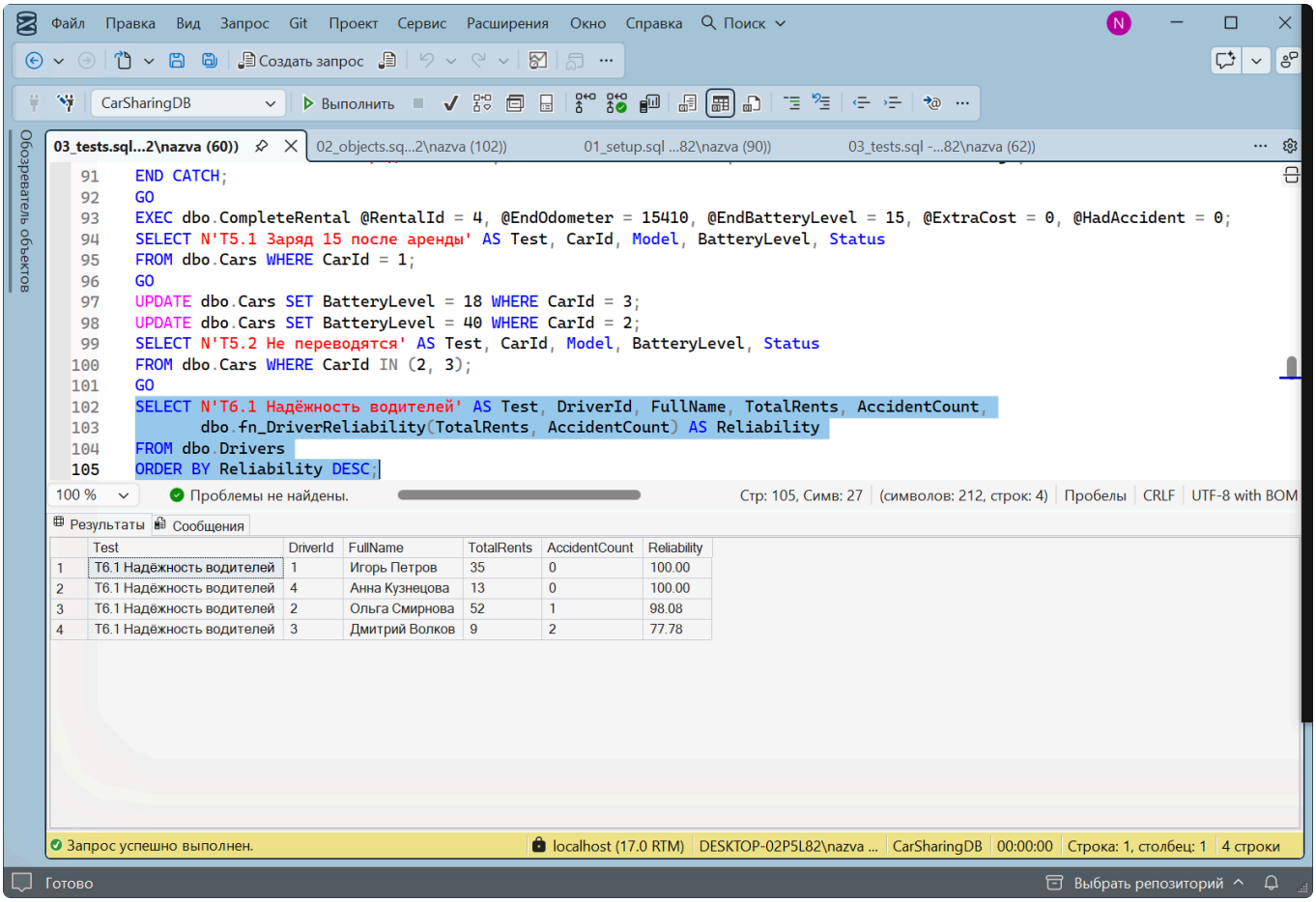
Тест

```
SELECT N'T6.1 Надёжность водителей' AS Test, DriverId, FullName, TotalRents,
AccidentCount,
```

```
        dbo.fn_DriverReliability(TotalRents, AccidentCount) AS Reliability
FROM dbo.Drivers
ORDER BY Reliability DESC;
```

Результат: Игорь Петров 100.00, Анна Кузнецова 100.00, Ольга Смирнова 98.08, Дмитрий Волков 77.78.

Скриншот 18. Надёжность водителей

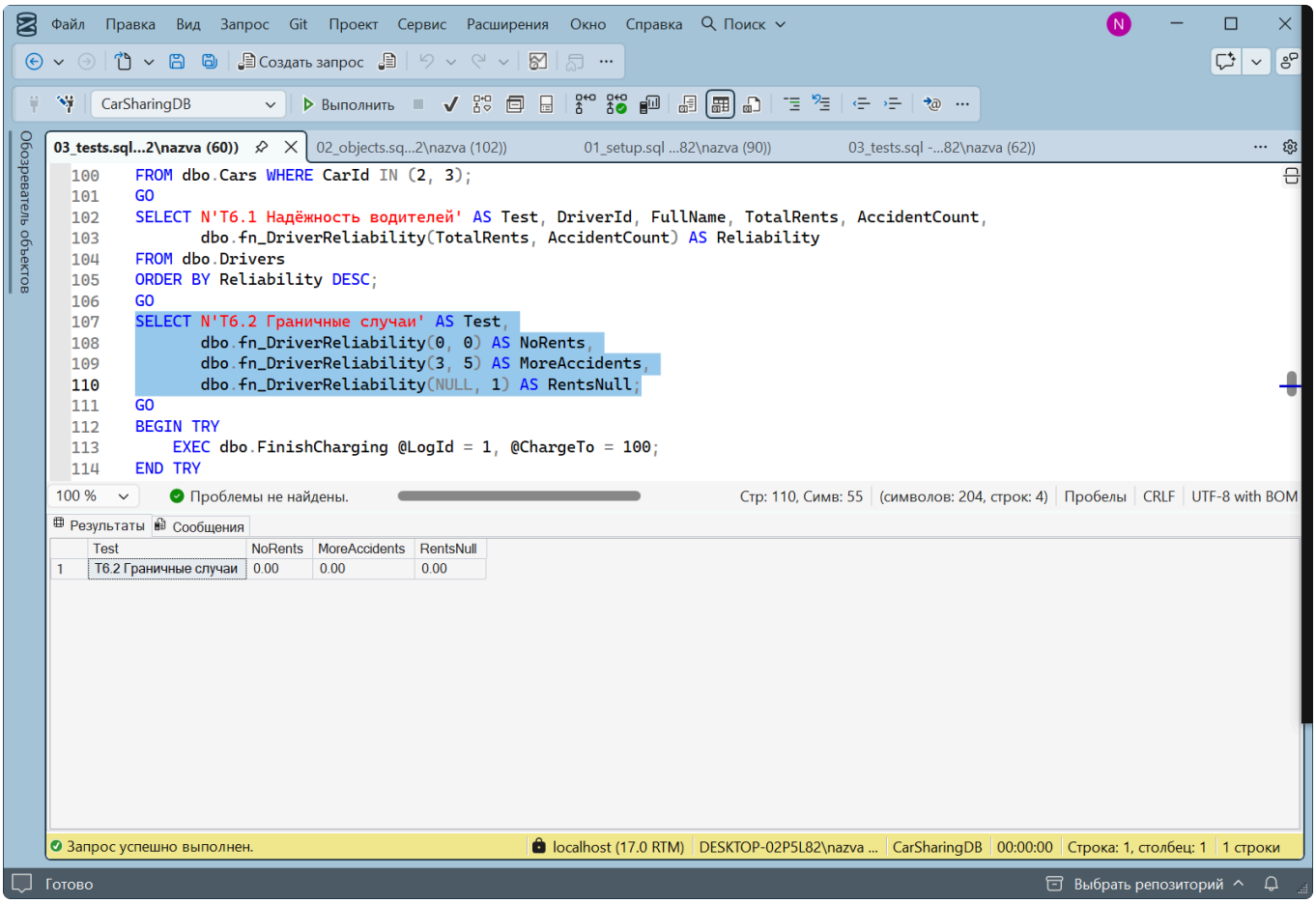


Тест

```
SELECT N'T6.2 Граничные случаи' AS Test,
       dbo.fn_DriverReliability(0, 0) AS NoRents,
       dbo.fn_DriverReliability(3, 5) AS MoreAccidents,
       dbo.fn_DriverReliability(NULL, 1) AS RentsNull;
```

Результат: Нет аренд - 0.00, аварий больше аренд - 0.00, NULL - 0.00.

Скриншот 19. Граничные случаи



Требование 7. Фиксация зарядки и восстановление заряда

Код

```
CREATE OR ALTER PROCEDURE dbo.FinishCharging
    @LogId       INT,
    @ChargeTo    INT,
    @ChargingEnd DATETIME = NULL
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;
    DECLARE @Msg NVARCHAR(400), @CarId INT, @ChargeFrom INT, @End DATETIME,
@Start DATETIME;
    BEGIN TRY
        BEGIN TRANSACTION;
            SELECT @CarId = CarId, @ChargeFrom = ChargeFrom, @End =
ChargingEnd, @Start = ChargingStart
            FROM dbo.ChargingLogs WITH (UPDLOCK, HOLDLOCK)
            WHERE LogId = @LogId;

            IF @CarId IS NULL
            BEGIN
```

```
                SET @Msg = CONCAT(N'Запись о зарядке LogId = ', @LogId, N' не
найдена.');
                THROW 52701, @Msg, 1;
            END;
            IF @End IS NOT NULL
            BEGIN
                SET @Msg = CONCAT(N'Зарядка LogId = ', @LogId, N' уже
завершена ', CONVERT(NVARCHAR(16), @End, 120), N'.');
                THROW 52702, @Msg, 1;
            END;
            IF @ChargeTo IS NULL OR @ChargeTo NOT BETWEEN 0 AND 100
            BEGIN
                SET @Msg = CONCAT(N'Некорректный уровень заряда ChargeTo = ',
@ChargeTo, N'.');
                THROW 52703, @Msg, 1;
            END;
            IF @ChargeTo < @ChargeFrom
            BEGIN
                SET @Msg = CONCAT(N'Уровень после зарядки (', @ChargeTo, N')
меньше уровня до зарядки (', @ChargeFrom, N').');
                THROW 52704, @Msg, 1;
            END;

            SET @ChargingEnd = ISNULL(@ChargingEnd, GETDATE());
            IF @ChargingEnd < @Start
            BEGIN
                SET @Msg = N'Время окончания зарядки раньше времени начала.';
                THROW 52705, @Msg, 1;
            END;

            UPDATE dbo.ChargingLogs
            SET ChargingEnd = @ChargingEnd,
                ChargeTo = @ChargeTo,
                EnergyConsumedKwh = @ChargeTo - ISNULL(@ChargeFrom, 0)
            WHERE LogId = @LogId;

            UPDATE dbo.Cars
            SET BatteryLevel = @ChargeTo,
                Status = CASE WHEN Status = N'Charging' THEN N'Available' ELSE
Status END
            WHERE CarId = @CarId;
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
```

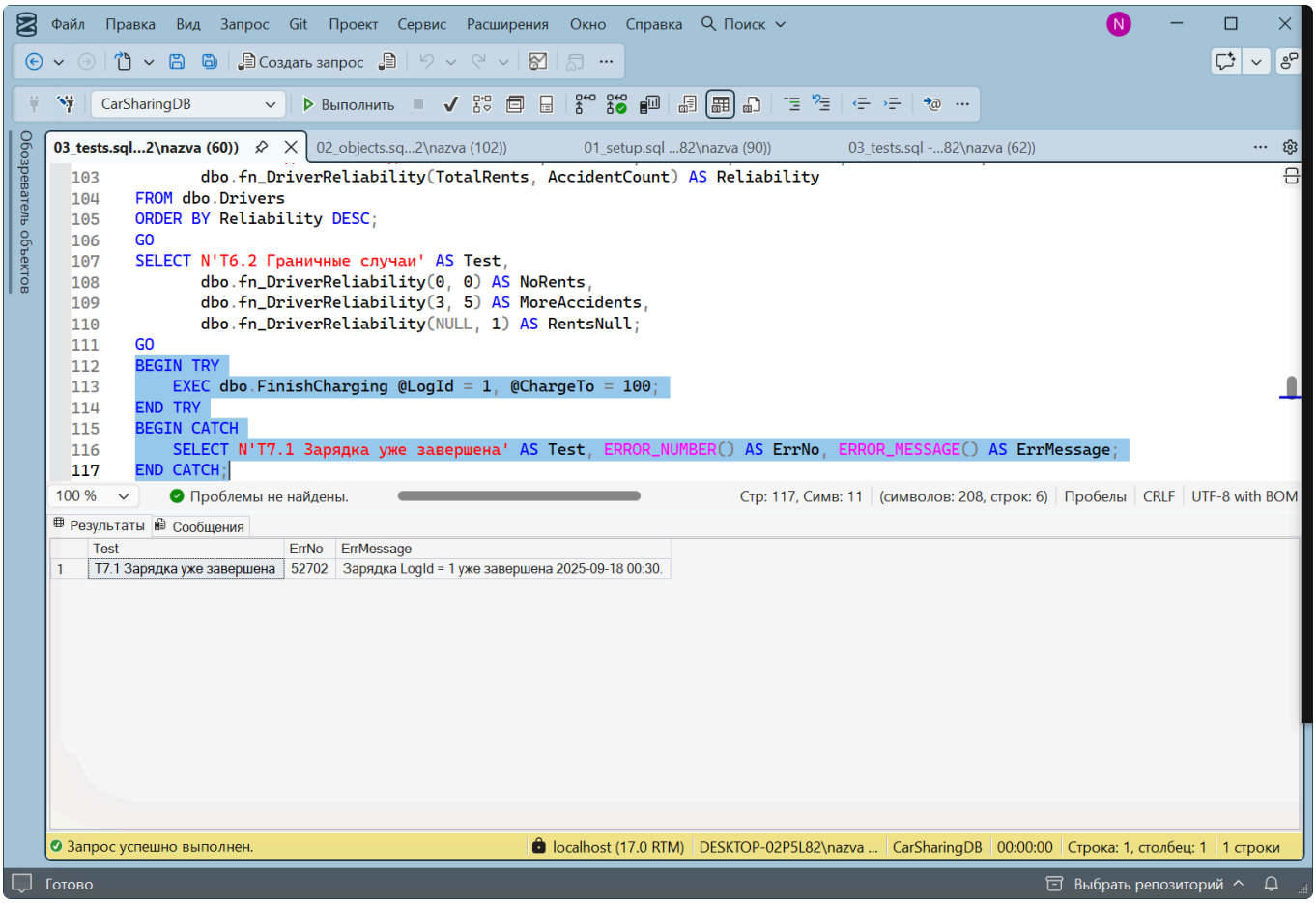
```
    END CATCH;
END;
```

Тест

```
BEGIN TRY
    EXEC dbo.FinishCharging @LogId = 1, @ChargeTo = 100;
END TRY
BEGIN CATCH
    SELECT N'T7.1 Зарядка уже завершена' AS Test, ERROR_NUMBER() AS ErrNo,
ERROR_MESSAGE() AS ErrMessage;
END CATCH;
```

Результат: Ошибка 52702 «Зарядка LogId = 1 уже завершена 2025-09-18 00:30.».

Скриншот 20. Зарядка уже завершена



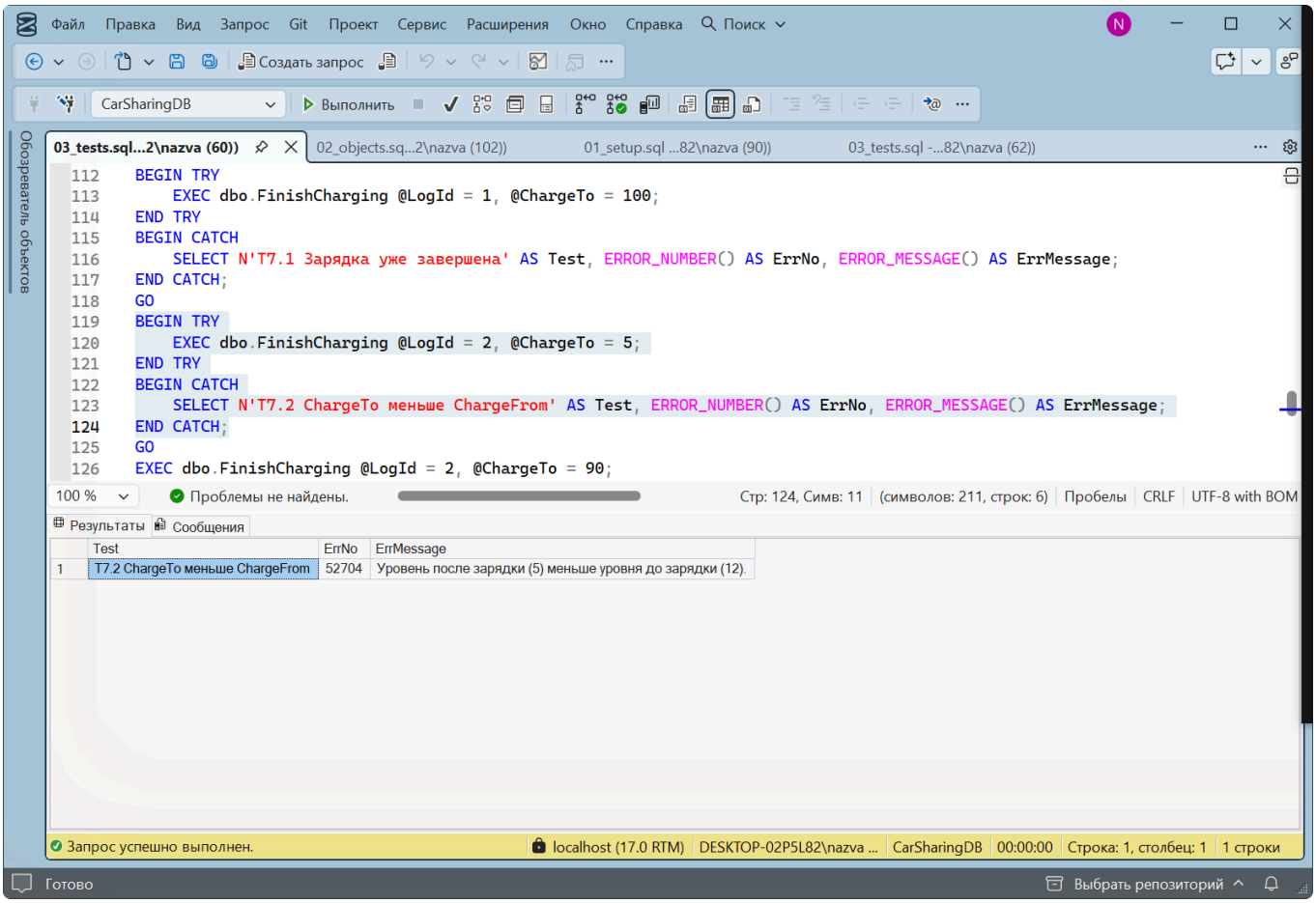
Тест

```
BEGIN TRY
    EXEC dbo.FinishCharging @LogId = 2, @ChargeTo = 5;
END TRY
```

```
BEGIN CATCH
    SELECT N'T7.2 ChargeTo меньше ChargeFrom' AS Test, ERROR_NUMBER() AS
ErrNo, ERROR_MESSAGE() AS ErrMessage;
END CATCH;
```

Результат: Ошибка 52704 «Уровень после зарядки (5) меньше уровня до зарядки (12).».

Скриншот 21. ChargeTo меньше ChargeFrom

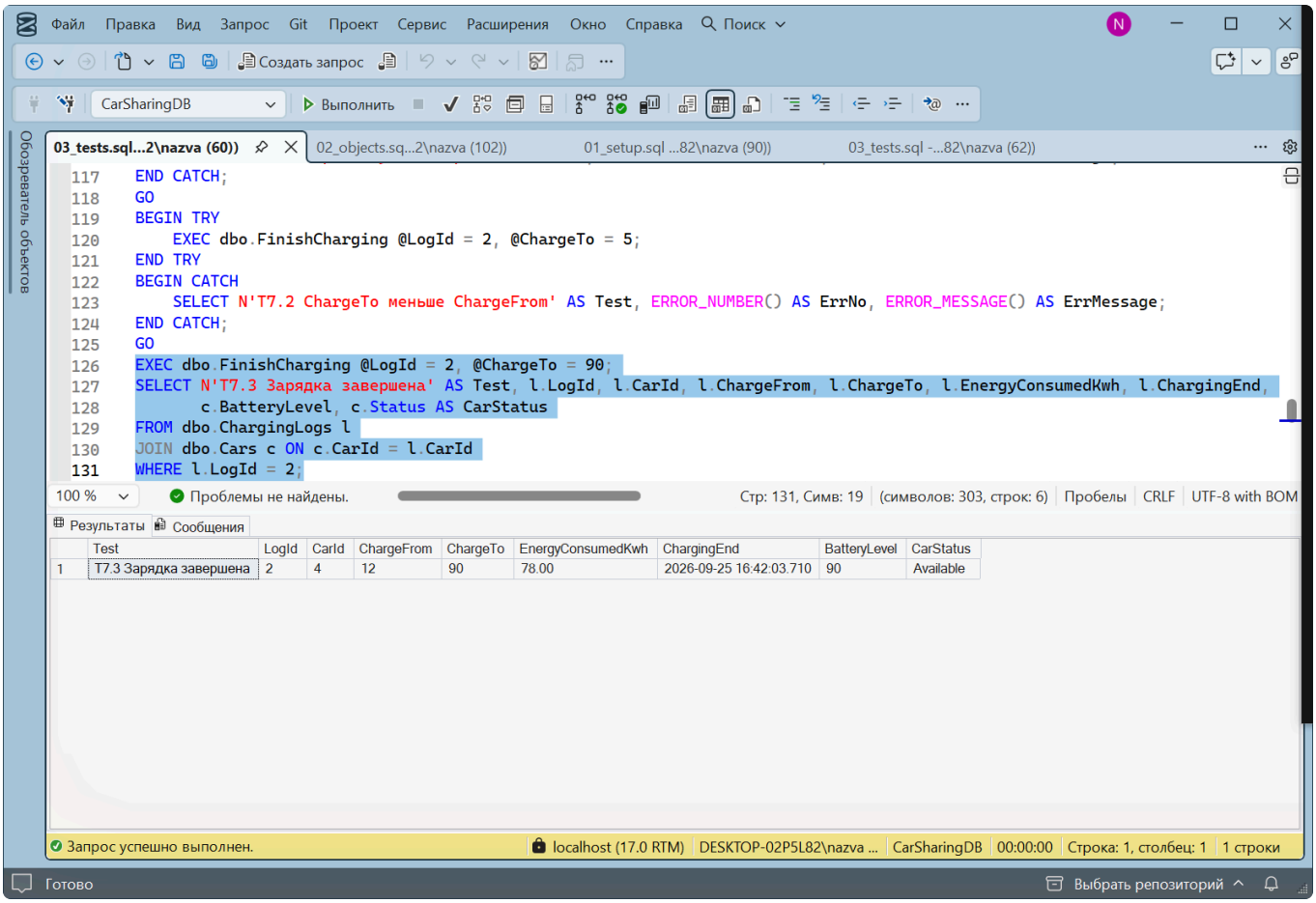


Тест

```
EXEC dbo.FinishCharging @LogId = 2, @ChargeTo = 90;
SELECT N'T7.3 Зарядка завершена' AS Test, l.LogId, l.CarId, l.ChargeFrom,
l.ChargeTo, l.EnergyConsumedKwh, l.ChargingEnd,
       c.BatteryLevel, c.Status AS CarStatus
FROM dbo.ChargingLogs l
JOIN dbo.Cars c ON c.CarId = l.CarId
WHERE l.LogId = 2;
```

Результат: Зарядка 2 завершена: ChargeFrom 12, ChargeTo 90, EnergyConsumedKwh 78.00, у Tesla Model 3 (CarId 4) заряд 90, статус Available.

Скриншот 22. Успешное завершение зарядки



Требование 8. Защита от аренды неисправного авто

Код

```
CREATE OR ALTER TRIGGER dbo.trg_Rentals_BlockUnavailableCar
ON dbo.Rentals
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @Msg NVARCHAR(400);
    IF NOT UPDATE(CarId) RETURN;

    IF EXISTS (SELECT 1
               FROM inserted i
               LEFT JOIN deleted d ON d.RentalId = i.RentalId
               JOIN dbo.Cars c ON c.CarId = i.CarId
               WHERE (d.RentalId IS NULL OR d.CarId <> i.CarId)
                 AND c.Status IN (N'Maintenance', N'Accident', N'Charging'))
    BEGIN
        SELECT TOP (1) @Msg = CONCAT(N'Автомобиль ', c.Model, N' (CarId = ',
```

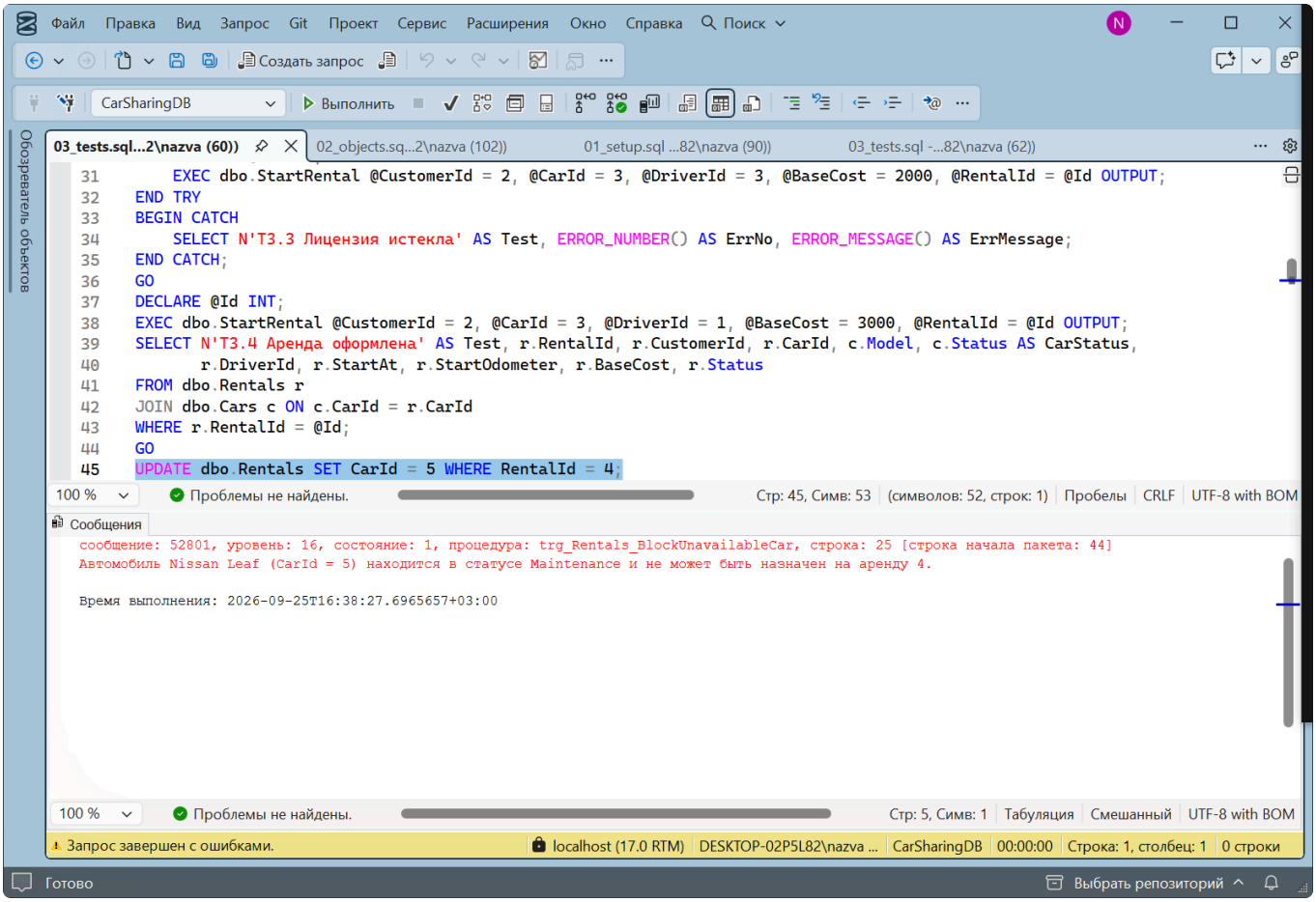
```
c.CarId, N') находится в статусе ', c.Status,
                                    N' и не может быть назначен на аренду ',
i.RentalId, N'.')
        FROM inserted i
        LEFT JOIN deleted d ON d.RentalId = i.RentalId
        JOIN dbo.Cars c ON c.CarId = i.CarId
        WHERE (d.RentalId IS NULL OR d.CarId <> i.CarId)
          AND c.Status IN (N'Maintenance', N'Accident', N'Charging');
        ROLLBACK TRANSACTION;
        THROW 52801, @Msg, 1;
    END;
END;
```

Тест

```
UPDATE dbo.Rentals SET CarId = 5 WHERE RentalId = 4;
```

Результат: Ошибка 52801 «Автомобиль Nissan Leaf (CarId = 5) находится в статусе Maintenance и не может быть назначен на аренду 4.».

Скриншот 9. Прямой UPDATE dbo.Rentals SET CarId = 5 WHERE RentalId = 4

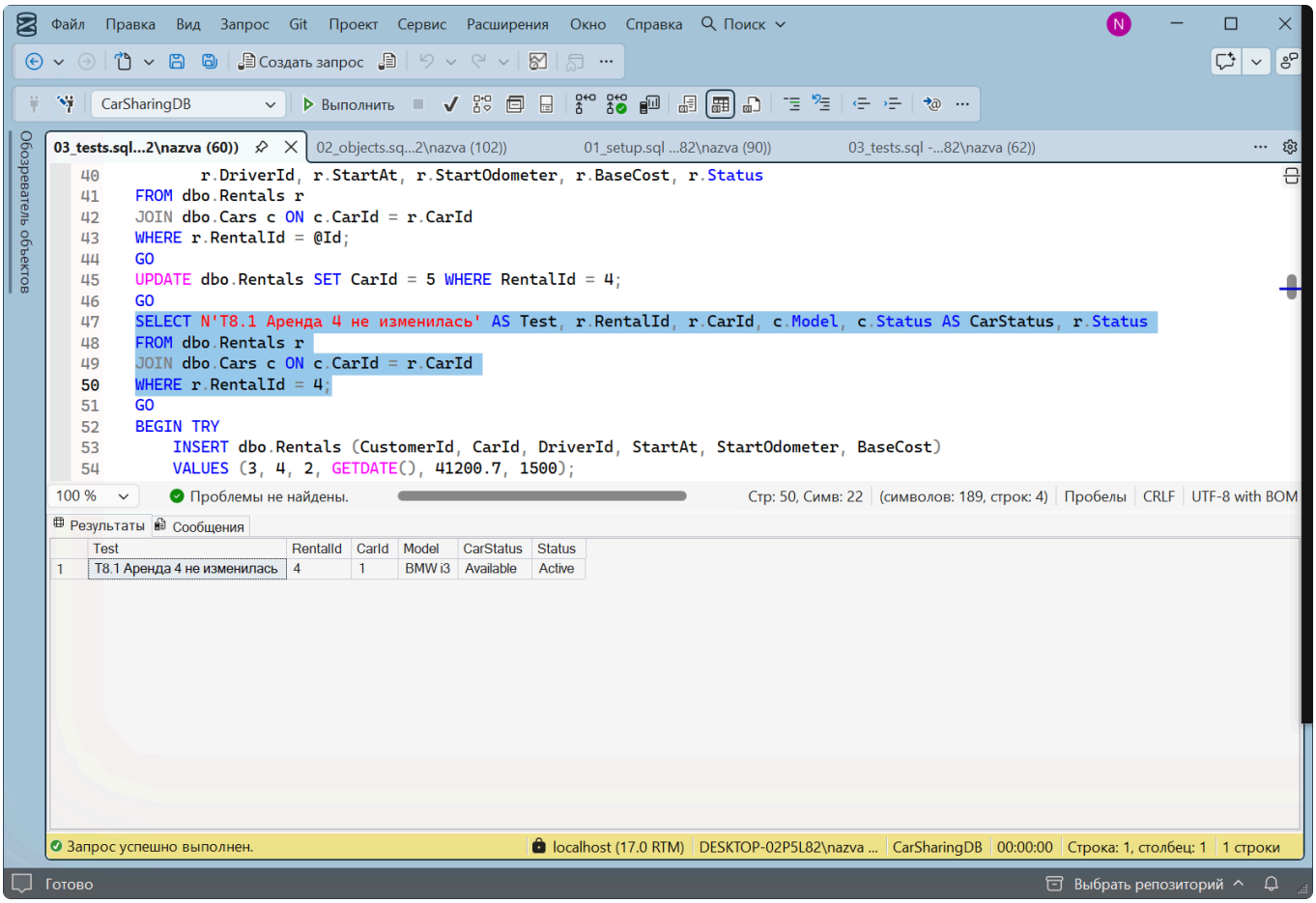


Тест

```
SELECT N'T8.1 Аренда 4 не изменилась' AS Test, r.RentalId, r.CarId, c.Model,
c.Status AS CarStatus, r.Status
FROM dbo.Rentals r
JOIN dbo.Cars c ON c.CarId = r.CarId
WHERE r.RentalId = 4;
```

Результат: Аренда 4 осталась на CarId 1, изменение откатилось.

Скриншот 10. Аренда 4 после отклонённого UPDATE



Тест

```
BEGIN TRY
    INSERT dbo.Rentals (CustomerId, CarId, DriverId, StartAt, StartOdometer,
BaseCost)
    VALUES (3, 4, 2, GETDATE(), 41200.7, 1500);
END TRY
BEGIN CATCH
    SELECT N'T8.2 INSERT с авто на зарядке' AS Test, ERROR_NUMBER() AS ErrNo,
```

```
ERROR_MESSAGE() AS ErrMessage;
END CATCH;
```

Результат: Ошибка 52801 «Автомобиль Tesla Model 3 (CarId = 4) находится в статусе Charging и не может быть назначен на аренду 6.».

Скриншот 11. Прямой INSERT с авто на зарядке

